GranMix: Adaptive Partition Granularity Management for Online Data Mixing

Abstract

Data mixing manages the allocation of limited pretraining tokens across heterogeneous corpora, thereby shaping the composition and training utility of the resulting data mixture. In most existing methods, source-based or clustered data groups are reweighted without adapting their granularity to regional sampling needs. However, coarse groups tie the sampling adjustments of their subsets together, partition should be fine where contribution differences justify independent weighting. Intuitively subdivide training data groups more finely enables useful subsets to receive independent sampling weights, yet uniform refinement also introduces redundant weights and amplifies estimation noise under limited feedback. We therefore present GranMix, an online data mixing framework that jointly adapts granularity and sampling weights of data groups, allowing coarse and fine groups to coexist. Our major contributions include (1) characterizing partition-induced control regret and forecasting the contributions of data groups with a bound on joint miss frequency; (2) designing a dynamic program for selecting tree cuts and target weights under a conservative objective and a group budget; (3) deriving a mixture handoff under coverage and distribution-change constraints with a lower bound on executed control gain; (4) evaluating GranMix's data mixtures on six SlimPajama workloads and four downstream tasks, comparing against uniform sampling and seven competitive methods, demonstrating effectiveness and utility of our methods.

ACM Reference Format:

. 2018. GranMix: Adaptive Partition Granularity Management for Online Data Mixing. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 20 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Training data management has become indispensable for language model training [6, 11]. However, as shown in recent data management research [9, 31, 45, 54], data subsets can differ in their usefulness in training. Assigning equal sampling weights to these subsets can therefore underuse informative data and limit the training gains achievable with a finite token budget. Data mixing addresses this allocation problem by selecting sampling proportions that improve pretraining effectiveness [5, 32, 50].

Most mixers assign one sampling weight to each source or group while keeping its within-group sampling distribution fixed, so a weight update changes the exposure of its examples together [18, 50]. These weights can be learned before target-model training [18, 32] or updated online [12, 17, 30]. Recent methods also reconsider the sampling units: CLIMB [15] and R&B [20] construct semantic groups through corpus-level clustering; DoGraph [51] revises gradient-based groups with a specified cluster count; and HERMES [40] provides hierarchical labels exposing multiple grouping resolutions. Yet these methods do not determine which corpus regions need finer sampling control and which can remain coarse.

In practice, training data's needs vary in granularity, from broad domain-level adjustments to targeted emphasis on particular languages or skills. For example, o1-mini [38]'s data mixture handles routine arithmetic but struggles with distractors [35]; Llama 3.3 70B [23] data mixture handles literal retrieval but struggles with two-hop associations [36]; Qwen2.5-Coder-32B [27] data mixture handles generates functions more reliably than reusing them [53]. In each case, under-trained capabilities are hidden within broader group, assigning only one weight to the broader group would prevent independently upweighting the gap-relevant subset. An intuitive solution is to subdivide every group as finely as possible, reducing these restrictions on selective sampling. However, this also creates redundant weights where similar adjustments suffice, potentially increasing estimation noise under limited feedback [30]

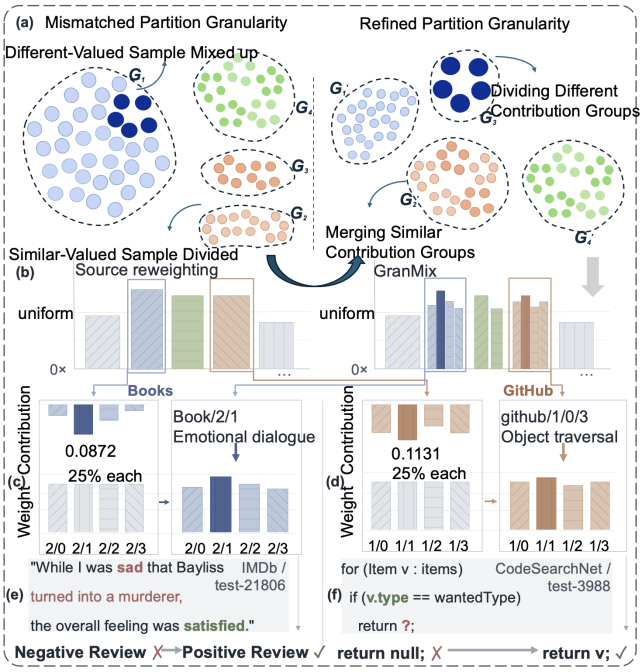


Figure 1: GranMix adapts granularity through selective splitting, merging, and subgroup reweighting.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

or the cost of obtaining reliable estimates [20]. A uniform granularity can therefore leave some regions too coarse for targeted adjustments and others unnecessarily fine, a granularity mismatch.

A useful insight to solve this mismatch comes from signal processing. To reconcile uneven regional demands, best-basis selection [14] constructs an adapted representation by comparing coarse components with combinations of finer alternatives at an additive cost. This suggests that by treating coarse groups and their independently weighted subgroups as alternative sampling representations, we can reconcile targeted adjustments where needed with shared control where separate weights are redundant. We therefore present GranMix, an online data mixing framework that jointly selects regional granularity and sampling weights. GranMix accumulates feedback across a hierarchy of candidate groups to forecast their contributions and calibrate prediction intervals. Using interval lower bounds and inherited sampling preferences, it compares shared and independent weighting under a common conservative objective. Deriving target weights analytically reduces partition selection to additive group scores, enabling a tree-cut dynamic program to combine coarse and fine groups under a shared group budget. Normalizing the selected scores yields the corresponding target weights. GranMix adjusts these weights to preserve coverage and limit distribution changes, and accepts an update only when its resulting conservative gain is positive.

Example 1. *Figure 1 combines a schematic of granularity mismatch with two cases from separate pretraining runs of a 160M model on the seven SlimPajama sources, each using 40k updates and initially assigning equal probability to the sources.*

Figure 1(b) contrasts changing a source's total sampling mass with changing the relative exposure of its subgroups. In the Books case (Figure 1(c)), book/2/1 *contains passages about emotions and attitudes, has an estimated contribution of* 0.0872 *at 2k updates, compared with* 0.0190–0.0471 *for its siblings. Source-level reweighting cannot exploit this distinction: increasing the Books weight raises their exposure together, leaving each child's conditional probability within* book/2 *at* 25%. *The restriction therefore lies in the partition, not merely in how accurately its weights are estimated. GranMix splits both* book *and* book/2, *allowing* book/2/1 *to receive an independent weight and increasing its conditional probability to* 28.8%. *The separate GitHub run in Figure 1(d) exhibits similar limitation for a subgroup dominated by Java/C# code, including attribute access and collection lookup. Its estimated contribution is* 0.1131, *above its siblings'* 0.0691–0.0921, *but a shared GitHub multiplier cannot preferentially expose the model to this subset. GranMix refines the source partition and raises the subgroup's probability within GitHub from* 25% *to* 27.08%. *In both cases, refinement converts an observed contribution difference into an available sampling adjustment, rather than merely increasing exposure to the entire source. At 40k updates, Books validation loss is* 3.8297 *versus* 3.8781 *for the probe-budget-matched uniform-source control, while GitHub validation loss is* 2.0596 *versus* 2.0823 *for Uniform.*

Figures 1(e) and 1(f) illustrate the final models' behavior under matched downstream adaptation. The IMDb example requires distinguishing sadness about a narrative event from the reviewer's overall evaluation: the control model predicts a negative label, whereas the GranMix-based model recognizes the positive review. The Java example requires returning an object once it satisfies the lookup condition; Uniform instead returns null, *while the GranMix-based model returns the matching object. The records establish the more immediate mechanism: recognizing a useful subset does not enable targeted exposure unless the partition makes that subset independently controllable. Conversely, the merge schematic in Figure 1(a) illustrates why independent control should not be introduced everywhere: where groups warrant similar adjustments, separate weights require additional estimates without providing a useful sampling distinction, and limited feedback can turn estimation noise into unnecessary weight differences. GranMix therefore refines where shared control hides useful differences and shares control where separate weights are redundant.*

1.1 Challenges

Jointly adapting regional partition granularity and sampling weights is challenging for three reasons:

(1) Whether subsets should share a sampling weight depends on their training contributions and inherited sampling preferences. However, contributions are observed only through selective sampling and change during training. Finer groups also have fewer observations, making it difficult to identify which differences justify independent weighting in the next window.

(2) The benefit of refining a region depends on the weights assigned to its subgroups, while additional groups consume the budget available to other regions. Partition and weight decisions are therefore coupled across the corpus, requiring a joint comparison of refinement opportunities under uncertain contributions and a shared group budget.

(3) Merging groups may remove sampling preferences that cannot be represented on the new partition. Moreover, target weights require adjustment to satisfy coverage and sample-distribution constraints. Such adjustments reduce the searched gain, so a favorable target mixture need not yield a beneficial executable update.

1.2 Contributions

Our major contributions are summarized as follows.

(1) We derive an explicit expression for partition-induced control regret and establish a necessary and sufficient condition for lossless grouping (Theorem 1), and develop an online diagnosis algorithm for contribution forecasting with a probability bound on joint prediction-miss frequency(Proposition 1), in Section 3.

(2) We design a budgeted refinement algorithm for joint partition and target-weight selection under a conservative objective, deriving closed-form target weights and an equivalent additive cut objective (Theorem 2) that enables dynamic programming over valid cuts of the fixed candidate tree, in Section 4.

(3) We devise a mixture handoff algorithm under coverage and sample-distribution KL constraints, characterize its feasibility and unique KL-projection solution (Theorem 3), and establish a lower bound on executed control gain and an upper bound on its average downside (Corollary 1), in Section 5.

(4) We evaluate GranMix on six SlimPajama workloads against competing mixers and matched fixed-cut controls in Section 6, demonstrating the benefit of regional granularity adaptation while assessing downstream performance and system cost.

Source code and proofs of theoretical results are provided in [1].

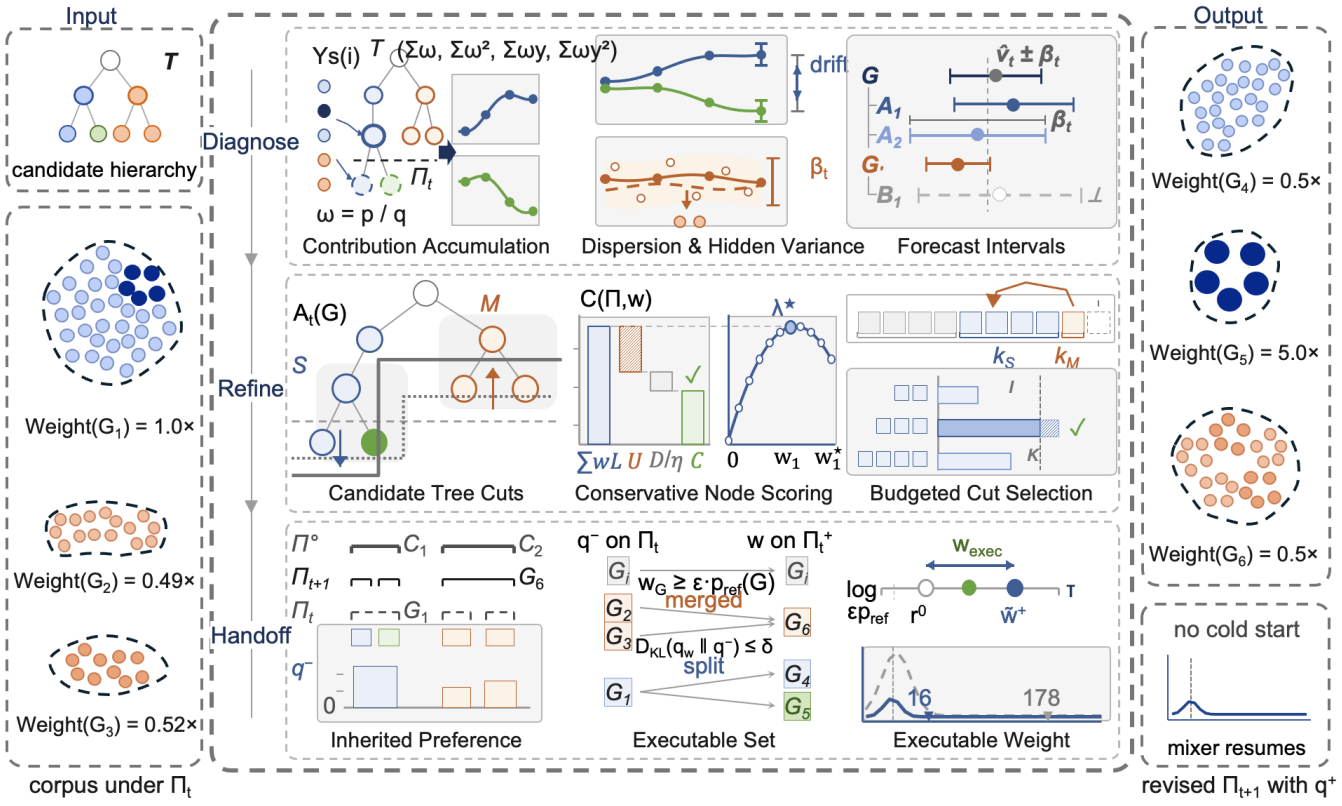


Figure 2: GranMix's online granularity management. Persistent node histories support diagnosis across cuts; candidate targets are converted to executable weights before the final update is selected.

2 GranMix Framework

We first introduce the sampling model, control objective, and admissible updates, then describe how GranMix combines diagnosis, refinement, and handoff to manage regional granularity online.

2.1 Preliminaries

Let $\mathcal{D}$ be a finite training corpus and $p(i) > 0$ a fixed reference distribution over its examples. Group-level sampling partitions $\mathcal{D}$ into disjoint groups $\Pi = \{G_1, \ldots, G_K\}$, with nonnegative weights satisfying $\sum_{G\in\Pi} w_G = 1$. For each group, write $p(G) = \sum_{i\in G} p(i)$ and $p(i \mid G) = p(i)/p(G)$. The sampler selects a group with probability w_G, then draws an example from $p(\cdot \mid G)$, inducing $q_{\Pi,w}(i) = w_G p(i \mid G), i \in G \in \Pi$. All examples in G share the multiplier $w_G/p(G)$, so reweighting adjusts their exposure together; splitting permits independent subset weights.

To restrict the partition search and give candidate groups persistent identities, we organize $\mathcal{D}$ into a fixed coarse-to-fine semantic tree $\mathcal{T}$. Its root contains $\mathcal{D}$, and the children of each internal node partition that node. A cut Π consists of disjoint nodes covering the corpus, subject to a group budget $|\Pi| \le m$. Selecting descendants refines a region; selecting their ancestor merges it. Different branches may be cut at different depths, so coarse and fine groups coexist. The tree supplies candidate boundaries, while training feedback determines which to use. A source domain may contain multiple active groups; unselected nodes remain candidates without independent weights, thus, m limits active weights.

Training proceeds in windows $\mathcal{W}_t$ of H optimizer steps. At boundary t, before $\mathcal{W}_t$ begins, Π_t and q_t^- denote the cut and sampler installed before the decision. Only observations through $\mathcal{W}_{t-1}$ are available, and the chosen configuration remains fixed throughout $\mathcal{W}_t$. For an example i from source domain $d(i)$, let $u_s(i)$ be its normalized first-order validation contribution at step s, with $|u_s(i)| \le M$ and larger values indicating more useful local updates. Section 3.1 derives this signal from the reduction in held-out validation loss $L_{V,d(i)}$ after one gradient step. Evaluated at the pre-update states encountered during $\mathcal{W}_t$, the window contributions are

$$v_t(i) = \frac{1}{H}\sum_{s\in\mathcal{W}_t} u_s(i), \qquad v_t(G) = \mathbb{E}_{p(\cdot|G)}[v_t(i)].$$

Group means use the fixed reference conditionals rather than observed sampling frequencies, providing a common reference across granularities. These window contributions are unknown at decision time and must be predicted from earlier observations.

The cut determines representable samplers; its weights allocate probability among groups. Their joint window control value is

$$J_t(q) = \sum_i q(i)v_t(i) - \frac{1}{\eta}D_{\mathrm{KL}}(q\|q_t^-). \tag{1}$$

The first term rewards local training contribution; the KL penalty charges for departure from the installed sampler q_t^-, with $\eta > 0$ controlling the tradeoff. All candidates share the same realized contribution field v_t, so $\Delta J_t(q) = J_t(q) - J_t(q_t^-)$ compares local sampling decisions on that field.

An executable update requires $w_G \geq \varepsilon p(G)$ for every selected group, with $0 < \varepsilon < 1$, and $D_{\mathrm{KL}}(q_{\Pi,w} \| q_t^-) \leq \delta$, where $\delta \geq 0$. The coverage floor preserves opportunities to observe every region, while the KL budget limits sample distribution changes caused by either weights or boundaries. Unlike the KL penalty used to rank candidates, the budget restricts which updates can be installed. The partition-control task is therefore to select a cut Π_t^+ with at most m groups and executable weights w_t^{exec}, using completed windows' evidence to seek an improvement in J_t. The decision may change the cut, reweight the current cut, or retain the installed sampler.

2.2 Online Granularity Management

Figure 2 shows GranMix's three-module loop. Shared state includes the fixed tree and reference masses, the installed sampling configuration, and histories indexed by candidate node and temporal block. Saved forecasts and their calibration state persist across decisions. Initialization uses a feasible cut under budget with reference weights, yielding $q_1^- = p$. Node histories start empty, and the calibration multiplier starts at $c_1 \geq 0$. Changing the active cut does not replace the underlying observation histories.

Diagnose. During training, the trainer supplies sample-level contributions and actual draw probabilities. The diagnoser accumulates importance-weighted observations along each sample's leaf-to-root path, including inactive candidates. At boundary t, it audits the record saved for $\mathcal{W}_{t-1}$, updates calibration, and publishes D_t and $\mathcal{V}_t$ for $\mathcal{W}_t$. Each valid-node record contains a forecast, interval, and latest observed normalized block signal; the separate snapshot $\mathcal{P}_t$ remains unchanged until that window is audited (Section 3).

Refine. Given the tree, current sampler, group budget, and valid-node intervals, refinement jointly selects a candidate cut and target weights under a conservative objective that discounts prediction uncertainty. It also reoptimizes weights on the current cut and sends both proposals to handoff before selection(Section 4).

Handoff. For each proposal, handoff accounts for inherited sampling preferences on proposed cut, while a lightweight mixer adjusts the target using the latest normalized block contribution signal. The solver returns weights satisfying the coverage floor and sample-distribution KL budget, or reports infeasibility. These executable weights are returned for final gain comparison (Section 5).

The controller compares feasible proposals using their conservative gains evaluated at the executable weights. It installs the proposal with the largest positive gain, preferring the current cut on ties; otherwise, it keeps the sampler unchanged. Adjustment is also deferred when any current-cut node lacks valid evidence. The selected configuration remains fixed throughout $\mathcal{W}_t$, while observations continue to accumulate for the next boundary.

3 Partition Diagnosis

The partition-control task in Section 2.1 requires a criterion for deciding which examples should share a sampling weight. Section 3.1 derives this criterion from local validation gains and the loss caused by grouping. Section 3.2 estimates the window contributions from training observations and calibrates their uncertainty.

3.1 Partition Criterion

At step s, let θ_s be the pre-update parameters, $g_s(i)$ the training gradient of example i, and $L_{V,d}$ the validation loss for domain d. Following the Taylor-based scoring of GREATS [49], write $g_{s,d}^V = \nabla L_{V,d}(\theta_s)$. A local step $\theta_s - h_s g_s(i)$ gives the validation gain $\Delta_s(i) = h_s \langle g_s(i), g_{s,d(i)}^V \rangle + R_s(i)$. If $L_{V,d}$ has an L_d-Lipschitz gradient along the update segment, then $|R_s(i)| \leq L_{d(i)} h_s^2 \|g_s(i)\|^2/2$. For $\nu > 0$, set $a_{s,d}^V = \max\{\|g_{s,d}^V\|, \nu\}$ and define the bounded signal

$$u_s(i) = \mathrm{clip}_M\left(\frac{\langle g_s(i), g_{s,d(i)}^V \rangle}{a_{s,d(i)}^V}\right), \tag{2}$$

where $\mathrm{clip}_M(x) = \min\{M, \max\{-M, x\}\}$. Normalization puts domain contributions on a common scale, and clipping limits the influence of individual observations. The trainer computes these scores before applying its batch optimizer update. Averaging over $\mathcal{W}_t$ gives $v_t(i)$; reference-weighted group averaging gives $v_t(G)$ for the control objective in Equation (1).

Independently estimating and weighting every sample of the training corpus is ideal but computationally forbidden and amplifies estimation noise; so samples are grouped together. This partition operation hides the weight control of the samples within groups, and thus a partition regret should be characterized.

For known v_t, omit the coverage floor and hard KL budget to isolate representation loss. Let $J_t^\star$ and $J_t^{\Pi,\star}$ be the maxima of J_t over all sample distributions and those representable on Π, respectively. Their difference is $\mathcal{R}_t(\Pi) = J_t^\star - J_t^{\Pi,\star}$. The sample preference $\phi_t(i) = \eta v_t(i) + \log(q_t^-(i)/p(i))$ combines contribution and inherited sampling preference; $\bar{\phi}_t(G) = \mathbb{E}_{p(\cdot|G)}[\phi_t(i)]$ is its reference-weighted mean within candidate group G.

Theorem 1 (Partition-induced control regret). *For positive q_t^- and p, every cut Π has partition regret*

$$\mathcal{R}_t(\Pi) = \frac{1}{\eta} \log \frac{\mathbb{E}_p[e^{\phi_t(i)}]}{\sum_{G \in \Pi} p(G) e^{\bar{\phi}_t(G)}} \geq 0. \tag{3}$$

Equality holds exactly when $\phi_t(i)$ is constant within every selected group. If q_t^-/p is constant within a group, this condition reduces to equal window contributions within that group.

Theorem 1 identifies two sources of useful sampling distinctions: window contributions and inherited preferences. Refinement exposes these distinctions, while merging assigns them a shared multiplier. When $q_t^- = p$ and η is small, the leading regret term is $\eta/2$ times the reference-weighted within-group contribution variance, relating the value of finer control to the variation it exposes. More generally, a unique interior optimizer $q_f^\star$ of a separably regularized objective is representable on a cut exactly when $q_f^\star/p$ is constant within each group. For a common strictly convex differentiable cost referenced to p, this means equal within-group contributions.

Candidate tree in Section 2.1 restricts the available distinctions under $|\Pi| \leq m$. The installed sampler supplies inherited preference; the diagnoser below estimates next-window node contributions.

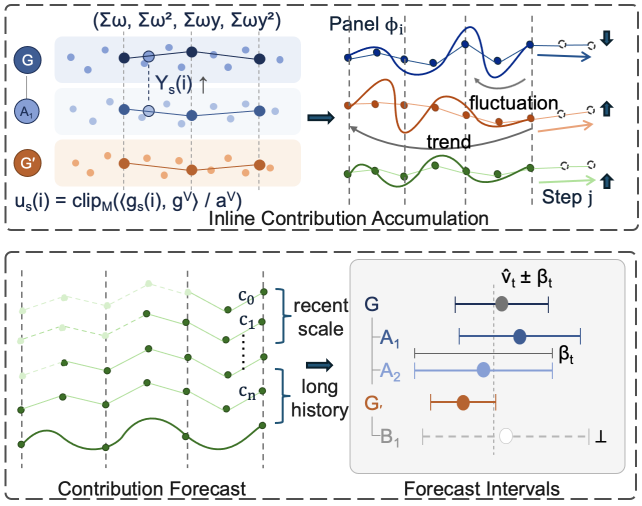


Figure 3: Online diagnosis converts persistent contribution histories into forecasts and calibrated intervals.

3.2 Diagnosis Algorithm

The diagnoser (Figure 3) forecasts contributions before a window and audits its saved predictions afterward to calibrate future widths. The audit also needs an interval because completed-window means remain only partially observed. To estimate means under p from samples drawn with actual probability q, each contribution $y = u_s(i)$ receives importance weight $\omega = p(i)/q$.

Use blocks of τ optimizer steps, with $\tau \mid H$. Before drawing a batch, the trainer fixes domain probes at θ_s, then computes sampled contributions using ghost inner products [49] before the optimizer update. The stream $\mathcal{O}_t$ records each draw in $\mathcal{W}_t$ as (i, y, q, b), including repetitions, where $b = \lfloor (s-1)/\tau \rfloor$. Along i's leaf-to-root path, it accumulates $(\omega, \omega^2, \omega y, \omega y^2)$ into $(W_{b,G}, W_{2,b,G}, S_{b,G}, Q_{b,G})$. These histories $\mathcal{S}$ include inactive nodes, persist across cut changes.

To accommodate changing contribution magnitudes, normalize each block by the scale at root r, $a_b = \max\{a_{\min}, \sqrt{Q_{b,r}/W_{b,r}}\}$, with $a_{\min} > 0$. This adjusts across time, complementing the domain normalization in Equation (2). Retain up to the latest $K_{\text{hist}} \geq H/\tau$ completed blocks in $\mathcal{B}_t$, keeping each window's blocks until its audit finishes. For the latest block $b_\star$, set $a_t = a_{b_\star}$ and $h_{t,b} = \lambda_{\text{hist}}^{b_\star - b}$, with $0 < \lambda_{\text{hist}} \leq 1$. For $W_t(G) = \sum_{b \in \mathcal{B}_t} h_{t,b} W_{b,G} > 0$, compute

$$\bar{z}_t(G) = \frac{\sum_{b \in \mathcal{B}_t} h_{t,b} S_{b,G}/a_b}{W_t(G)}, \qquad \hat{v}_t(G) = \text{clip}_M\big(a_t \bar{z}_t(G)\big).$$

To account for histories dominated by large weights, use the effective count $n_{\text{eff},t}(G) = W_t(G)^2 / \sum_{b \in \mathcal{B}_t} h_{t,b}^2 W_{2,b,G}$. Let $s_t(G)$ be the weighted standard deviation of y/a_b with weights $h_{t,b}\omega$, and $s_t^{\text{blk}}(G)$ that of nonempty block means $S_{b,G}/(a_b W_{b,G})$ with weights $h_{t,b} W_{b,G}$; both use population moments. Recent drift is $d_t(G) = |\bar{z}_t^{\text{recent}}(G) - \bar{z}_t(G)|$, using the same weighting over the latest $1 \leq K_{\text{recent}} \leq K_{\text{hist}}$ blocks and setting $d_t(G) = 0$ if empty. Sampling variability, block variation, and drift define

$$b_t(G) = b_{\min} + a_t \left(\lambda_1 \frac{s_t(G)}{\sqrt{n_{\text{eff},t}(G)}} + \lambda_2 s_t^{\text{blk}}(G) + \lambda_3 d_t(G) \right),$$

where $b_{\min} > 0$ and $\lambda_1, \lambda_2, \lambda_3 \geq 0$. With all the variables above, the prediction radius is $\beta_t(G) = c_t b_t(G)$, with shared multiplier $c_t \geq 0$, giving $L_t(G) = \hat{v}_t(G) - \beta_t(G)$ and $U_t(G) = \hat{v}_t(G) + \beta_t(G)$.

The routine $\text{Forecast}_t(G, \mathcal{S})$ returns the forecast, base scale, effective count, and mixer signal $m_t(G) = S_{b_G,G}/(a_{b_G} W_{b_G,G})$. Here b_G is the latest retained block with $W_{b_G,G} > 0$, so $m_t(G)$ is its normalized mean contribution for G. Empty histories return $(\perp, \perp, 0, \perp)$. Nodes with $n_{\text{eff},t}(G) \geq n_{\min} > 0$ and finite outputs form $\mathcal{V}_t$, with records $D_t(G) = (\hat{v}_t(G), [L_t(G), U_t(G)], m_t(G))$. Before $\mathcal{W}_t$, save $\mathcal{P}_t = (\mathcal{V}_t, \mathcal{H}_t, c_t)$, where $\mathcal{H}_t(G) = (\hat{v}_t(G), b_t(G))$, so the audit checks predictions fixed before the window.

After $\mathcal{W}_t$, node means remain partially observed, so calibration needs an audit estimate. Let $N = HB$ count all draws, with B per step, and let $S_{t,G}^{\text{win}}$ sum $S_{b,G}$ over the window, taking zero for unobserved nodes. Use $\check{v}_t(G) = \text{clip}_M\big(S_{t,G}^{\text{win}}/(Np(G))\big)$. Its fixed denominator estimates the completed window's reference mean from contributions before division by a_b; forecasting instead self-normalizes historical weights. Both reuse the training log.

Allocating $\delta_{\text{aud}}/[t(t+1)]$ to window t, where $\delta_{\text{aud}} \in (0,1)$, and taking a union bound over $\mathcal{T}$, the two-sided Bernstein–Freedman bound [19] gives $r_t^{\text{aud}}(G) = M \min\{2, 2x/3 + \sqrt{2x + 4x^2/9}\}$, where $x = x_t(G) = \log(2|\mathcal{T}|t(t+1)/\delta_{\text{aud}})/(N\varepsilon p(G))$. Clipping $\check{v}_t(G) \pm r_t^{\text{aud}}(G)$ to $[-M, M]$ gives $[\check{L}_t(G), \check{U}_t(G)]$. These intervals simultaneously cover all nodes and windows with probability at least $1 - \delta_{\text{aud}}$; only the saved valid nodes need auditing.

On audit coverage, containment of the audit interval implies coverage by the saved prediction interval. Set $\text{err}_t = 1$ if containment fails for any $G \in \mathcal{V}_t$, and zero otherwise, including an empty valid set. Following adaptive conformal inference [22], update

$$c_{t+1} = \max\{0, c_t + \gamma_{\text{cal}}(\text{err}_t - \zeta)\}. \tag{4}$$

Here, $\zeta \in (0,1)$ is target failure rate and $\gamma_{\text{cal}} > 0$ is step size. Failed checks increase multiplier; successful checks reduce it toward zero.

Proposition 1 (Online joint coverage). *Under the audit conditions above, let z_t indicate a prediction miss for at least one $G \in \mathcal{V}_t$. Set $C_{\max} = \max\{c_1, 2M/b_{\min} + \gamma_{\text{cal}}\}$. With probability at least $1 - \delta_{\text{aud}}$, simultaneously for every $T \geq 1$, the joint miss frequency satisfies*

$$\frac{1}{T} \sum_{t=1}^{T} z_t \leq \zeta + \frac{C_{\max} - c_1}{\gamma_{\text{cal}} T}. \tag{5}$$

On simultaneous audit coverage, $z_t \leq \text{err}_t$. Since prediction centers and audit endpoints lie in $[-M, M]$ and $b_t(G) \geq b_{\min}$, all checks pass once $c_t \geq 2M/b_{\min}$, keeping $c_t \leq C_{\max}$. Summing Equation (4) gives the bound. It controls the frequency of windows with any valid-node miss, including nodes selected by refinement.

Algorithm 1 accumulates $\mathcal{O}_t$, audits the saved predictions, and publishes records for $\mathcal{W}_{t+1}$. Initialize with empty histories and $\mathcal{P}_1 = (\varnothing, \varnothing, c_1)$, where $c_1 \geq 0$. All diagnostic settings are fixed before training except the online multiplier.

4 Partition Refinement

Beyond partition regret, uncertainty in contribution estimates and execution constraints can introduce additional losses relative to independent sample control. Uncertain estimates may lead to unnecessary splits, excessive weights for low-contribution subgroups, or inappropriate merges of subgroups with distinct contributions.

Algorithm 1: Online Partition Diagnosis and Calibration

Input: Tree $\mathcal{T}$; reference p; records O_t for $\mathcal{W}_t$; saved record $\mathcal{P}_t$; diagnostic settings
Maintained state: Node–block history $\mathcal{S}$, with missing entries zero
Output: Valid-node records D_{t+1}, set $\mathcal{V}_{t+1}$, and snapshot $\mathcal{P}_{t+1}$

foreach *record* $(i, y, q, b) \in O_t$, *in order* **do**
  $\omega \leftarrow p(i)/q$
  foreach *node* G *on the leaf-to-root path of* i **do**
    $\mathcal{S}_{b,G} \leftarrow \mathcal{S}_{b,G} + (\omega, \omega^2, \omega y, \omega y^2)$
Read $\mathcal{V}_t, c_t$ and recover $[L_t, U_t]$ from $\mathcal{P}_t$
Form the audit intervals $[\check{L}_t, \check{U}_t]$ on $\mathcal{V}_t$ from the completed-window totals
$\mathrm{err}_t \leftarrow \mathbf{1}\{\exists G \in \mathcal{V}_t : [\check{L}_t(G), \check{U}_t(G)] \not\subseteq [L_t(G), U_t(G)]\}$
$c_{t+1} \leftarrow \max\{0, c_t + \gamma_{\mathrm{cal}}(\mathrm{err}_t - \zeta)\}$
$(D_{t+1}, \mathcal{V}_{t+1}, \mathcal{H}) \leftarrow (\varnothing, \varnothing, \varnothing)$
foreach *node* $G \in \mathcal{T}$ **do**
  $(\hat{v}, b, n, m_G) \leftarrow \mathrm{Forecast}_{t+1}(G, \mathcal{S})$
  if $n \geq n_{\min}$ *and all four quantities are finite* **then**
    $D_{t+1}(G) \leftarrow (\hat{v}, [\hat{v} - c_{t+1}b, \hat{v} + c_{t+1}b], m_G)$
    $\mathcal{V}_{t+1} \leftarrow \mathcal{V}_{t+1} \cup \{G\}$; $\mathcal{H}(G) \leftarrow (\hat{v}, b)$
$\mathcal{P}_{t+1} \leftarrow (\mathcal{V}_{t+1}, \mathcal{H}, c_{t+1})$
return $(D_{t+1}, \mathcal{V}_{t+1}, \mathcal{P}_{t+1})$

Meanwhile, maintaining minimum sampling coverage and bounding changes in the sampling distribution restrict feasible updates. In this section, we use the forecasts and intervals from Section 3 to construct conservative node scores and jointly select a target partition and its weights under the group budget. The execution constraints are enforced through mixture handoff in Section 5.

4.1 Refinement Criterion

For a candidate $q = q_{\Pi,w}$, the loss relative to independent sample control decomposes as $J^\star - J(q) = \mathcal{R}(\Pi) + [J^{\Pi,\star} - J(q)]$. The first term measures the cost of grouping from previous section; the second measures the loss from the chosen weights, including the effects of prediction uncertainty and execution constraints. Useful refinement therefore combines finer resolution with weights that can be reliably estimated and executed.

Let $U^- = \sum_{G\in\Pi^-} q^-(G)U(G)$ be the installed sampler's upper contribution estimate. We use node lower bounds for a candidate and this upper bound for the installed sampler, giving

$$C(\Pi, w) = \sum_{G\in\Pi} w_G L(G) - U^- - \eta^{-1} D_{\mathrm{KL}}(q_{\Pi,w} \| q^-). \tag{6}$$

The lower bound $L(G) = \hat{v}(G) - \beta(G)$ discounts prediction uncertainty. Define the excess error $e(G) = (|v(G) - \hat{v}(G)| - \beta(G))_+$. Its effect on the comparison is $E(\Pi, w) = \sum_{G\in\Pi} w_G e(G) + \sum_{G\in\Pi^-} q^-(G)e(G)$, weighting interval misses under both the candidate and installed samplers.

Proposition 2 (Conservative gain and excess prediction error). *Every candidate cut and weight vector satisfies*

$$\Delta J(q_{\Pi,w}) \geq C(\Pi, w) - E(\Pi, w). \tag{7}$$

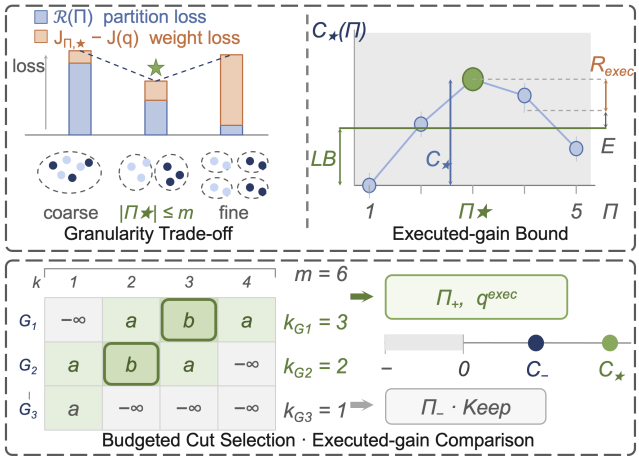


Figure 4: Budgeted refinement balances partition and weight losses before comparing executable gains.

When the intervals cover both candidate and current-cut contributions, $E(\Pi, w) = 0$, and a positive conservative gain implies an improvement in the window control objective.

The bound weights interval misses by the sampling mass assigned to the affected nodes. Together with Proposition 1, it relates forecast reliability to the quality of the selected sampling decision.

Let $\kappa(G) = \mathbb{E}_{p(\cdot|G)}[\log(q^-(i)/p(i))]$ be the inherited log preference. Replacing $v(G)$ by $L(G)$ in $\tilde{\phi}(G) = \eta v(G) + \kappa(G)$ gives a conservative preference; maximizing $C(\Pi, w)$ over w for a fixed cut then yields the additive scores in Theorem2. The inherited term is available directly from the installed cut. For an active group B, write $q^-(i)/p(i) = q^-(B)/p(B)$, so every descendant G has $\kappa(G) = \log(q^-(B)/p(B))$. At an ancestor, $\kappa(G)$ is the reference-mass-weighted average of its children's values. Propagating these values through the tree gives inherited preferences at all candidate resolutions from the current group weights.

Theorem 2 (Target weights and additive scores). *For positive q^- and p, finite node intervals, and $\eta > 0$, define node score*

$$A(G) = p(G) \exp\{\eta L(G) + \kappa(G)\}. \tag{8}$$

For any valid cut Π, set $w^\star_G = A(G)/\sum_{B\in\Pi} A(B)$. Every weight vector w on the group-weight simplex satisfies

$$C(\Pi, w) \leq C^\star(\Pi) = \eta^{-1} \log \sum_{G\in\Pi} A(G) - U^-, \tag{9}$$

with equality if and only if $w = w^\star$.

Let $\mathcal{C}$ contain valid cuts $\Pi \subseteq \mathcal{V}$ of $\mathcal{T}$ with $|\Pi| \leq m$. Since U^- is common to all candidates and the logarithm is increasing, joint optimization of partitions and target weights reduces to

$$\Pi^\star \in \arg\max_{\Pi\in\mathcal{C}} \sum_{G\in\Pi} A(G). \tag{10}$$

Each score combines reference mass, the calibrated contribution bound, and inherited sampling preference. The same scores determine the cut and recover its target weights by normalization.

For a parent G with children $G_1, \ldots, G_d$, selecting the parent contributes $A(G)$, whereas selecting all children contributes $\sum_j A(G_j)$ and uses $d-1$ additional groups. Each node has its own forecast and radius, so the children's lower bounds need not average to the parent's. Wider child intervals reduce their exponential scores, while distinct, well-supported contributions can make their combined score larger. The cut objective therefore evaluates the contribution gained by introducing independent weights together with the evidence supporting those weights. Their group cost is handled by the shared budget in the dynamic program.

4.2 Refinement Algorithm

As shown in Figure 4, we solve Equation (10) by bottom-up dynamic programming. For $1 \le k \le m$, let $F(G,k)$ be the maximum score of a cut covering the subtree rooted at G with exactly k valid nodes. A cut either selects G or combines cuts of all its child subtrees. Initialize all entries to $-\infty$, then set $F(G,1) = A(G)$ when $G \in \mathcal{V}$. For children $G_1, \ldots, G_d$, let $\mathcal{K}_d(k)$ contain the positive integer allocations $\mathbf{k} = (k_1, \ldots, k_d)$ that sum to k. Update

$$F(G,k) \leftarrow \max\left\{F(G,k), \max_{\mathbf{k}\in\mathcal{K}_d(k)} \sum_{j=1}^{d} F(G_j,k_j)\right\}. \tag{11}$$

An invalid node cannot be selected, but valid descendants may cover its subtree. We combine child tables sequentially and record the maximizing allocations for backtracking.

For example, consider a tree of valid nodes whose root has children A and B, each with two leaves, and a budget of three groups. Its three-group state compares $F(A,2) + F(B,1)$ with $F(A,1) + F(B,2)$. The first refines A and keeps B coarse; the second reverses that allocation. Backtracking follows the maximizing choice to recover a mixed-resolution cut. The root also compares its one- and two-group states, so the extra group is used when it improves the total score. For a deeper tree, each child table already summarizes its own resolution choices, and the same convolution combines them.

At the root r, maximize $F(r,k)$ over $1 \le k \le m$, preferring fewer groups on ties. Backtracking returns $\Pi^\star$, and Theorem 2 gives its target weights. For the fixed tree, valid nodes, and group budget, this target-cut search takes $O(|\mathcal{T}|m^2)$ time and $O(|\mathcal{T}|m)$ space. The two subsequent handoff calls add their execution costs.

The controller evaluates two proposals: the searched cut and the current cut with reoptimized target weights. Algorithm 3 converts each to executable weights before comparison. Among feasible proposals, it installs the one with the largest positive C, preferring the current cut on ties. Otherwise, Keep preserves the installed cut and weights; an invalid current-cut node also triggers Keep.

Here $\Theta_{\text{exec}} = (\eta_{\text{mix}}, \varepsilon, \delta, \xi)$ and $\boldsymbol{m}_\Pi = (m(G))_{G\in\Pi}$ comes from D. The valid installed cut supplies a feasible root state since $|\Pi^-| \le m$. Lines 9–11 combine child tables and store maximizing allocations for backtracking. Lines 15–19 recover target weights, apply handoff, and compare the resulting executable gains.

Algorithm 2: Budgeted Partition Refinement

Input: Tree $\mathcal{T}$ with root r; reference p; installed cut Π^- and sampler q^-; records D, set $\mathcal{V}$; budget m, η, and Θ_{exec}
Output: Update (Π^+, w^{exec}), or Keep

1 **if** $\Pi^- \notin \mathcal{V}$ **then return** Keep
2 $A(G) \leftarrow p(G)e^{\eta L(G)+\kappa(G)}$, $G \in \mathcal{V}$
3 $F(G, 1{:}m) \leftarrow -\infty$ for all G
4 **foreach** *node G in postorder* **do**
5 **if** $G \in \mathcal{V}$ **then** $F(G,1) \leftarrow A(G)$; mark selecting G
6 **if** *G has children* **then**
7 $F_{\text{tmp}}(0) \leftarrow 0$, $F_{\text{tmp}}(1{:}m) \leftarrow -\infty$
8 **foreach** *child G' of G, in a fixed order* **do**
9 $F_{\text{new}}(0) \leftarrow -\infty$; for $1 \le k \le m$, set $F_{\text{new}}(k) \leftarrow \max_{1\le j\le k}\{F_{\text{tmp}}(k-j) + F(G',j)\}$
10 Store a maximizing j in $J_{G,G'}(k)$ for each finite entry; $F_{\text{tmp}} \leftarrow F_{\text{new}}$
11 For $1 \le k \le m$, replace $F(G,k)$ by $F_{\text{tmp}}(k)$ and mark combining children when strictly larger
12 $k^\star \leftarrow \min\arg\max_{1\le k\le m} F(r,k)$; $\Pi^\star \leftarrow$ Backtrack$(r, k^\star)$
13 $(\text{choice}, g_{\text{choice}}) \leftarrow (\text{Keep}, 0)$
14 **foreach** *distinct cut Π in the ordered list* $(\Pi^-, \Pi^\star)$ **do**
15 $w^\star_G \leftarrow A(G)/\sum_{B\in\Pi} A(B)$, $G \in \Pi$
16 $w^{\text{exec}} \leftarrow$ Handoff$(\Pi, w^\star, \boldsymbol{m}_\Pi, q^-, p; \Theta_{\text{exec}})$
17 **if** $w^{\text{exec}} \ne$ Infeasible **then**
18 $g \leftarrow C(\Pi, w^{\text{exec}})$ by Equation (6)
19 **if** $g > g_{\text{choice}}$ **then** $(\text{choice}, g_{\text{choice}}) \leftarrow ((\Pi, w^{\text{exec}}), g)$
20 **return** choice

5 Mixture Handoff

While Handing off the target weight and partition to execute, two constrains are necessary: a coverage floor to preserve sampling opportunities for every data region, allowing continued contribution estimation even for regions currently assigned low weights; and a constraint limits distribution changes, promoting smooth transitions between consecutive control windows rather than abrupt shifts in the training-data mixture. In this section, we develop a mixture handoff that converts the proposed targets into executable weights satisfying both constraints.

For a proposed cut Π^+, let $w^\star$ be the target from Theorem 2, define $q_w(i) = w_G p(i \mid G)$ for $i \in G \in \Pi^+$. We require $w_G \ge \varepsilon p(G)$ and $D_{\text{KL}}(q_w \| q^-) \le \delta$. Coverage floor ensures $q_w(i) \ge \varepsilon p(i)$ and bounds importance weights by $1/\varepsilon$. The inherited reference weights are $r^0(G) = p(G)e^{\kappa(G)}/Z$, with $Z = \sum_{G\in\Pi^+} p(G)e^{\kappa(G)}$. Here $\kappa(G)$ is log preference from Section 4.1. Sample-level divergence $K(w) = D_{\text{KL}}(q_w \| q^-)$ reduces to

$$K(w) = D_{\text{KL}}(w \| r^0) - \log Z. \tag{12}$$

On the proposed cut, r^0 minimizes the divergence from the installed sampler before imposing coverage floors. The term $-\log Z \ge 0$ is minimum cost of representing installed preferences on this cut.

When the proposed cut refines the installed cut, $q^-(i)/p(i)$ is constant within every new group. Thus $p(G)e^{\kappa(G)} = q^-(G)$, giving $Z = 1$ and inherited weights equal to the old sampler's masses on the refined groups. A merge instead replaces the old multipliers by their reference-weighted geometric mean. If it joins groups with different multipliers, Jensen's inequality gives $Z < 1$, and $-\log Z$ charges for the sampling distinctions lost by merging. This cost is present before adjusting the new weights toward the mixer target, explaining why the minimum-change endpoint determines whether a proposed structure fits the KL budget.

Algorithm 3: Executable Mixture Handoff

Input: Cut Π^+; target $w^\star$; signals $\boldsymbol{m}_{\Pi^+}$; installed sampler q^-; reference p; Θ_{exec}, with $0 < \xi < 1$
Output: Executable weights w^{exec}, or Infeasible

1 $Z \leftarrow \sum_{G\in\Pi^+} p(G)e^{\kappa(G)}$; $r^0(G) \leftarrow p(G)e^{\kappa(G)}/Z$
2 $\widetilde{w} \leftarrow \text{Normalize}(\{w^\star_G e^{\eta_{\text{mix}} m(G)}\}_G)$
3 $\ell_G \leftarrow \varepsilon p(G)$; $w^0 \leftarrow \text{FloorNormalize}(r^0, \ell)$
4 **if** $K(w^0) > \delta$ **then return** Infeasible
5 $w^1 \leftarrow \text{FloorNormalize}(\widetilde{w}, \ell)$
6 **if** $K(w^1) \le \delta$ **then return** w^1
7 **if** $K(w^0) = \delta$ **then return** w^0
8 $(a, b) \leftarrow (0, 1)$; $w^{\text{exec}} \leftarrow w^0$
9 **while** $b - a > \xi$ **do**
10 $\quad \alpha \leftarrow (a + b)/2$
11 $\quad w \leftarrow \text{FloorNormalize}(\{\widetilde{w}^\alpha_G r^0(G)^{1-\alpha}\}_G, \ell)$
12 $\quad$ **if** $K(w) \le \delta$ **then**
13 $\quad\quad a \leftarrow \alpha$; $w^{\text{exec}} \leftarrow w$
14 $\quad$ **else**
15 $\quad\quad b \leftarrow \alpha$
16 **return** w^{exec}

The target $w^\star$ optimizes the conservative objective. We then construct a mixer based on contributions. At the window boundary, the mixer favors groups with larger contributions for the next window, using $\widetilde{w}_G \propto w^\star_G e^{\eta_{\text{mix}} m(G)}$. Here $m(G) = m_t(G)$ is the latest normalized block mean from Section 3.2, and $\eta_{\text{mix}} \ge 0$ controls this additional tilt. Let $\ell_G = \varepsilon p(G)$. The executable set $\mathcal{E}(\Pi^+)$ contains unit-sum weights with $w_G \ge \ell_G$ and $K(w) \le \delta$. Handoff solves

$$w^{\text{exec}} = \arg\min_{w\in\mathcal{E}(\Pi^+)} D_{\text{KL}}(w\|\widetilde{w}). \tag{13}$$

Theorem 3 (Executable mixture weights). *Assume positive q^-, p, and $\widetilde{w}$, and $0 < \varepsilon < 1$. For $\alpha \in [0, 1]$, define*

$$w_G(\alpha) = \max\left\{\ell_G, \frac{\widetilde{w}^\alpha_G r^0(G)^{1-\alpha}}{Z_\alpha}\right\}, \tag{14}$$

where $Z_\alpha > 0$ makes the weights sum to one. The feasible set is nonempty when $K(w(0)) \le \delta$. When nonempty, unique solution is $w^{\text{exec}} = w(\alpha^\star)$, $\alpha^\star$ is the largest $\alpha \in [0, 1]$ satisfying $K(w(\alpha)) \le \delta$.

For positive z and $\sum_G \ell_G < 1$, FloorNormalize(z, ℓ) returns $w_G = \max\{\ell_G, z_G/Z_{\text{free}}\}$ with unit total mass. Starting from $S = \varnothing$, it computes $Z_{\text{free}} = (\sum_{G\notin S} z_G)/(1 - \sum_{G\in S} \ell_G)$, fixes entries with $z_G/Z_{\text{free}} < \ell_G$ at their floors, and adds them to S. It repeats until all remaining entries satisfy their floors. Since $K(w(\alpha))$ is nondecreasing, Algorithm 3 checks endpoints and bisects the geometric path in Equation (14), including the representation cost in every KL test.

The geometric path follows from combining the two KL terms. A multiplier $\lambda \ge 0$ for the change budget gives the objective $D_{\text{KL}}(w\|\widetilde{w}) + \lambda D_{\text{KL}}(w\|r^0)$, up to terms constant in w. Writing $\alpha = 1/(1+\lambda)$, stationarity above the floors gives $w_G \propto \widetilde{w}^\alpha_G r^0(G)^{1-\alpha}$. Applying the floors yields Equation (14), with $\alpha = 0$ minimizing change and $\alpha = 1$ giving the floor-constrained mixer target.

The endpoint tests handle an infeasible cut, a feasible target, and the minimum-budget case $\delta = K(w^0)$. Otherwise, lines 8–16 maintain $a \le \alpha^\star \le b$ and return the feasible lower endpoint $w(a)$ once $b - a \le \xi$. Thus ξ controls the search interval in α. For $K = |\Pi^+|$, floor-normalization costs $O(K^2)$, and bisection takes $O(\log(1/\xi))$ evaluations. In finite precision, the returned weights are checked for normalization, coverage floors, and KL feasibility.

For any feasible returned vector, the loss relative to the original target is $R^{\text{exec}} = C^\star(\Pi^+) - C(\Pi^+, w^{\text{exec}})$, with the exact identity

$$R^{\text{exec}} = \eta^{-1} D_{\text{KL}}(w^{\text{exec}}\|w^\star). \tag{15}$$

This identity measures the combined effect of the mixer and execution constraints relative to the pre-mixer target $w^\star$.

Rearranging gives $C(\Pi^+, w^{\text{exec}}) = C^\star(\Pi^+) - R^{\text{exec}}$. A proposal retains positive conservative gain when its searched gain exceeds the loss from mixing and projection. Different proposals can incur different execution losses, so handoff can change ranking. Algorithm 2 therefore compares both proposals at their returned weights.

Corollary 1 (Gain after handoff). *A feasible proposal with the target $w^\star$ from Theorem 2 satisfies $\Delta J(q^{\text{exec}}) \ge C^\star(\Pi^+) - R^{\text{exec}} - E(\Pi^+, w^{\text{exec}})$. Set $q^{\text{exec}}_t = q^-_t$ on Keep. With probability at least $1 - \delta_{\text{aud}}$, the following bound holds simultaneously for every $T \ge 1$:*

$$\frac{1}{T}\sum_{t=1}^{T}[-\Delta J_t(q^{\text{exec}}_t)]_+ \le 4M\left(\zeta + \frac{C_{\max} - c_1}{\gamma_{\text{cal}} T}\right). \tag{16}$$

Corollary 1 separates the searched gain, execution loss, and prediction errors beyond the calibrated radii. Because Algorithm 2 installs only feasible proposals with positive gain, calibrated interval misses control the average downside in the window objective.

6 Experiments

We evaluate GranMix by (1) comparing pretraining quality and efficiency with existing mixers, (2) assessing regional granularity, diagnostic reliability, and execution constraints through matched controls and ablations, and (3) examining downstream applications under matched adaptation.

Our results show that: (1) GranMix achieves the best mean pretraining performance on all workloads and outperforms the tested fixed-cut controls, supporting the effectiveness of adaptive granularity; (2) GranMix improves mean downstream performance over uniform mixing on both generation and classification tasks under matched adaptation, demonstrating its practical utility.

6.1 Experimental Settings

All runs use one four-RTX-4090 node. The main backbone is a 160M-parameter decoder-only transformer with a 2,048-token context and 128 sequences per global batch (262,144 tokens/update). Unless varied, GranMix uses $m = 16$, statistics blocks of $\tau = 20$ steps, and control windows of $H = 100$ steps on subsets or $H = 200$ on seven sources; cuts and weights stay fixed within each window.

Each subset window contains five statistics blocks; each seven-source window contains ten, giving 50 and 200 windows in the respective training budgets. Contributions use actual draw probabilities for importance weights $p(i)/q(i)$ and accumulate along leaf-to-root paths, including inactive nodes. Completed-window evidence and Section 3.2's eligibility checks govern decisions; insufficient current-cut evidence retains the installed sampler.

6.1.1 Datasets. We use five SlimPajama subsets—A/SE, GH/C4, B/SE, A/B/SE, and CC/GH/W—and all seven sources. A, B, GH, SE,

Table 1: Equal-source geometric test perplexity on six SlimPajama workloads.

Method	A/SE		GH/C4		B/SE		A/B/SE		CC/GH/W		SlimPajama (7)	
Uniform	16.647		36.526		47.574		35.480		41.942		25.647	
DoReMi	16.589	(-0.058)	37.237	(+0.711)	47.081	(-0.493)	35.190	(-0.290)	42.872	(+0.930)	27.587	(+1.940)
MixMin	16.467	(-0.180)	36.552	(+0.026)	47.723	(+0.149)	35.468	(-0.012)	41.693	(-0.249)	25.521	(-0.126)
RegMix	16.515	(-0.132)	36.290	(-0.236)	48.530	(+0.956)	35.268	(-0.212)	41.847	(-0.095)	27.926	(+2.279)
ADO	16.624	(-0.023)	37.089	(+0.563)	47.909	(+0.335)	35.446	(-0.034)	41.841	(-0.101)	25.671	(+0.024)
DGA / source-m	16.672	(+0.025)	36.762	(+0.236)	47.791	(+0.217)	35.422	(-0.058)	42.457	(+0.515)	25.667	(+0.020)
DGA / semantic16	16.535	(-0.112)	36.342	(-0.184)	46.956	(-0.618)	35.176	(-0.304)	42.253	(+0.311)	25.375	(-0.272)
R&B / source-m	16.460	(-0.187)	36.641	(+0.115)	47.246	(-0.328)	35.168	(-0.312)	43.080	(+1.138)	25.527	(-0.120)
R&B / semantic16	16.379	(-0.268)	36.460	(-0.066)	49.217	(+1.643)	37.298	(+1.818)	43.477	(+1.535)	26.565	(+0.918)
DoGraph	16.695	(+0.048)	36.091	(-0.435)	49.133	(+1.559)	35.529	(+0.049)	41.347	(-0.595)	27.564	(+1.917)
GranMix	16.186	(-0.461)	35.576	(-0.950)	46.876	(-0.698)	35.153	(-0.327)	41.082	(-0.860)	25.198	(-0.449)

CC, and W denote ArXiv, Books, GitHub, StackExchange, CommonCrawl, and Wikipedia; C4 is the seventh source. Subsets use 5,000 updates (1.31072B tokens); seven-source runs use 40,000 (10.48576B). Training, controller-probe, development, and test documents are disjoint before packing, keeping the controller's observations separate from the documents used for final evaluation.

6.1.2 Candidate hierarchy. The candidate hierarchy uses only training documents, embedded with jina-v2[24] into 512-dimensional L_2-normalized vectors and partitioned by capacity-constrained recursive spherical k-means within each source. Probe, development, and test documents are excluded. Children partition their parent, so a valid cut covers the corpus exactly once. The seven-source tree contains 224 leaves (32 per source) in 445 nodes, with at most 16 active groups. An ancestor shares a weight across descendants, while finer cuts permit independent weights at varying depths.

The reference sampler assigns probability $1/D$ to each of the D sources. Each of source d's N_d packed examples has $p(i) = 1/(DN_d)$; node masses sum these reference probabilities. Setting $w_G = p(G)$ yields $q(i) = p(i)$ on any valid cut, preserving the reference mixture. Cuts retain node identities and histories, so newly active groups reuse prior observations.

6.1.3 Baselines. DoReMi [50], MixMin [47], and RegMix [32] learn fixed mixtures. ADO [30], DGA [17], and R&B [20] update weights online; DoGraph [51] also revises gradient-based groups. DGA and R&B keep their weighting procedures with source-m (one group per source) or semantic16 (16 fixed semantic groups). Uniform samples sources equally and packed examples uniformly within sources; GranMix and matched controls use it for initialization.

6.1.4 Criteria. We report equal-source geometric test perplexity, $\mathrm{PPL}_{\mathrm{eq}} = \exp(D^{-1}\sum_{d=1}^{D}\mathcal{L}_d)$, for source d's token-mean loss $\mathcal{L}_d$ and D sources. Lower is better. Results average three independent pretraining seeds; matched controls include standard deviations. A node-hour is an elapsed hour on the four-GPU node. GranMix's accounted time includes contribution measurement, online control, and a 1,800-second setup allowance. Downstream exact match (EM) and accuracy differences are absolute percentage points.

6.2 Comparing with Existing Methods

Table 1 gives mean test PPL and signed differences from Uniform. GranMix is best on all six workloads, improving over Uniform by 0.92–2.77%. On seven sources, DGA/semantic16 reaches 25.375 (−0.272), versus GranMix's 25.198, another 0.177 lower. Margins over the strongest competitors are 0.015 on A/B/SE (R&B/source-m) and 0.080 on B/SE (DGA/semantic16).

Semantic16 improves DGA on every workload but R&B only on A/SE and GH/C4. On B/SE, it lowers DGA's PPL by 0.835 and raises R&B's by 1.971: the benefit depends on the weighting procedure.

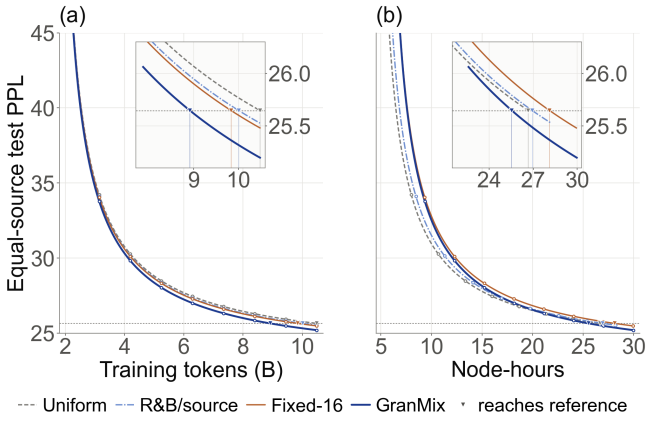


Figure 5: Quality versus training tokens and accounted node-hours on the seven-source workload.

Figure 5 compares quality against (a) training tokens in billions and (b) accounted node-hours. The dotted line marks Uniform's final PPL (25.647), triangles mark when methods reach it, and insets enlarge endpoints. GranMix reaches this reference after 34,000 versus 40,000 updates, saving 15.0% of tokens. Including control and the setup allowance, it takes 25.55 versus 26.67 node-hours (4.2% less). At 40,000 updates, it attains 25.198 PPL in 29.97 node-hours, exceeding Uniform's runtime at this fixed token budget.

At 410M parameters on seven sources, mean test PPL over three seeds improves from Uniform's 24.878 to 24.425 (1.82%); development PPL falls from 24.760 to 24.225.

6.3 Evaluating Our Methods

Fixed-Source retains the source cut; Fixed-m fixes a legal m-node cut by balancing reference masses before training (Fixed-16 by

default). Both adapt weights and share GranMix's tree, candidate-wide histories, predictor, calibration, weight solver, handoff, and initial sampler, isolating the effect of adaptive partition selection.

Table 2: Matched fixed-cut controls: test perplexity and mean active groups on A/B/SE and seven sources.

	A/B/SE		Seven sources	
Variant	PPL	$\overline{K}$	PPL	$\overline{K}$
Fixed-Source	35.331 ± .047	3.0	25.576 ± .023	7.0
Fixed-16	35.237 ± .041	16.0	25.481 ± .021	16.0
GranMix	35.153 ± .032	12.4	25.198 ± .018	12.9

Table 2 gives three-seed PPL as mean ± standard deviation and time-averaged active-group count $\overline{K}$. Fixed-16 improves over Fixed-Source. GranMix further lowers PPL by 0.084 on A/B/SE and 0.283 on seven sources, using 12.4 and 12.9 groups: adaptive cuts improve mean PPL with fewer weights than Fixed-16.

Across 45 continuation snapshots, executable structural proposals improve held-out loss over Keep in 34 cases and executable current-cut reweighting in 29. Their one-window advantage over reweighting averages 0.0008 nats across these snapshots.

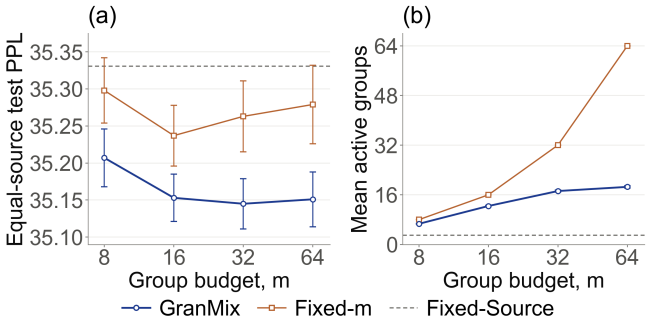


Figure 6: Pretraining quality and active-group counts with varying group budgets on A/B/SE.

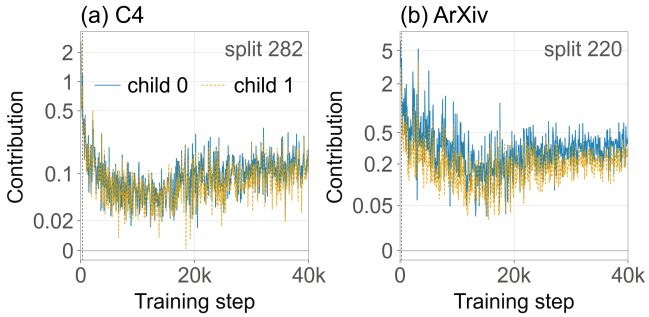


Figure 7: Sibling-group contribution traces for C4 and ArXiv over 40,000 seven-source pretraining steps.

Figure 6 varies $m \in \{8, 16, 32, 64\}$ on A/B/SE: panel (a) gives three-seed mean PPL with whiskers showing seed variation, and panel (b) gives the time-averaged active-group count. Dashed lines denote the three-group Fixed-Source control. GranMix outperforms Fixed-m at every budget: PPL improves from 35.207 at $m = 8$ to 35.153 at $m = 16$, with little further change. Fixed-m uses m groups throughout; GranMix remains below the larger limits. The budget therefore permits, rather than mandates, finer control, without prescribing a uniform tree depth.

CPU-only search medians are 16–1,100 ms across 63–4,095 nodes at $m = 16$. For 1,023 nodes, $m = 8 \rightarrow 32$ raises time 74 → 1,045 ms and memory 0.351 → 1.288 MiB, consistent with $O(|\mathcal{T}|m^2)$. These CPU measurements isolate cut search from contribution measurement and model training.

Figure 7 follows siblings child 0 and child 1 over 40,000 steps in the C4 and ArXiv cases labeled "split 282" and "split 220". Larger estimated contributions favor increased sampling, not higher loss; the nonlinear vertical scales differ between panels. C4's traces largely overlap, whereas ArXiv's separate during parts of later training. Shared control may thus suffice in one region but conceal useful distinctions in another. These signals alone do not determine splits: refinement also considers uncertainty and inherited preferences.

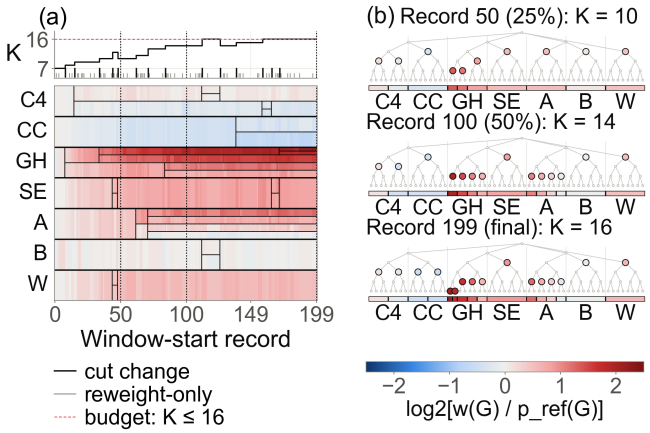


Figure 8: Evolution of active groups and relative sampling weights during seven-source pretraining.

Figure 8(a) plots $K = |\Pi|$ against the dashed $K = 16$ budget. Active groups within each source are colored by $\log_2[w(G)/p(G)]$, with $p_{\text{ref}} = p$. Red/blue means increased/decreased exposure; zero means reference-level exposure, not an unchanged sampler. Black and gray ticks mark cut and weight-only changes, respectively. Record r denotes step $200r$, from initialization at record 0 to the final training window at step 39,800.

Panel (b) shows selected subtrees with active/unselected nodes colored/gray. Record 100 (20,000 steps) has four groups each in ArXiv and GitHub, two in C4, and one per remaining source ($K = 14$); the final window has 16. The 200 records contain 14 cut changes, 58 weight-only updates, and 128 Keep records including initialization, leaving 199 between-window transitions.

Diagnosis uses twenty 1,000-window streams with contributions bounded by M and known means, including shared-scale changes, persistent shifts, and temporary fluctuations. All controllers forecast before observing samples. Last block uses the latest block

with calibration; Fixed multiplier uses full-history prediction with a frozen multiplier; Full calibrated uses both. Shared fields and sampling budgets yield distinct histories under each sampler.

Table 3: Forecast accuracy, interval reliability, and update frequency on bounded streams with known means.

Diagnoser	Error	Miss (%)	Width	Update (%)
Last block	0.061	6.7	0.227	21.9
Fixed multiplier	0.037	18.2	0.151	38.7
Full calibrated	0.035	4.3	0.194	31.2

In Table 3, Error is mean absolute forecast error divided by M; Width is mean interval width divided by $2M$. Miss counts windows with any valid-node true mean outside its saved prediction interval; Update counts installed cut or weight changes. Both rates include all evaluated windows, including Keep decisions, rather than conditioning on accepted updates.

Full calibrated has the lowest error and joint-miss rate. Relative to Fixed multiplier, width rises from 0.151 to 0.194 while misses fall from 18.2% to 4.3% and updates from 38.7% to 31.2%. Last block has wider intervals and fewer updates (21.9%).

Eligibility is checked per node from its importance-weighted history; a well-sampled source can still have descendants with insufficient evidence. Prediction intervals are saved before each window and audited only after it ends. Both miss and audit rates therefore concern forecasts issued before the evaluated observations.

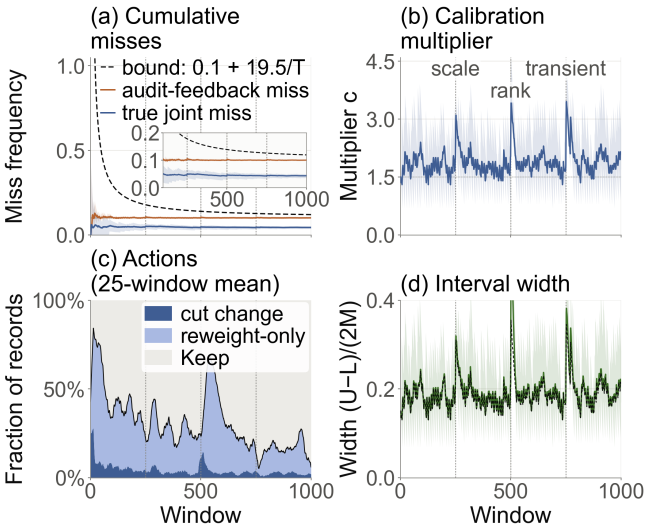


Figure 9: Forecast intervals, audit feedback, and controller actions on bounded streams with known contribution means.

Figure 9(a) plots true joint misses and audit-feedback failures against Proposition 1's bound $0.1+19.5/T$. An audit interval extending beyond its saved prediction interval triggers an audit failure, not necessarily a true-mean miss. At $\zeta = 0.1$, $T = 1{,}000$, the twenty-stream rates are 4.3% and 10.04%, below the bound 0.1195. Nodewise coverage is 98.6%; only 0.6% of intervals span the full range.

Panel (b) shows c_t for shared-scale shifts, rank changes, and transient fluctuations. Panel (c) gives cut-change, weight-only, and Keep fractions averaged over 25 windows; panel (d) normalizes prediction-interval widths by the contribution range $2M$.

Table 4: Controller ablations on A/B/SE.

Variant	PPL ↓	Cut	Weight	Keep
GranMix	35.153	12	14	24
Point estimates	35.407	27	16	7
Fixed calibration	35.265	18	16	16
No KL budget	35.301	19	14	17

Table 4 uses the same A/B/SE budget over 50 window-start records including initialization. Cut, Weight, and Keep denote partition changes, reweighting only, and no change. Point estimates collapses intervals but retains eligibility filtering; Fixed calibration freezes the multiplier; No KL budget removes the hard constraint but keeps coverage floors, the KL penalty, and executable-gain testing. All three raise PPL. Removing the hard budget raises the 95th-percentile sample-level KL change from 0.050 to 0.137.

Table 5: Time spent on ordinary training and online controller components during seven-source pretraining.

Component	Time (s)	Share (%)
Domain validation probes	6,800	6.41
Ghost inner products	2,400	2.26
Statistics and reduction	760	0.72
Forecast, audit, calibration	45	0.04
Scoring and cut search	23	0.02
Two handoffs and installation	52	0.05
Ordinary training and input	96,000	90.50
Total online run	106,080	100.00

Table 5 excludes setup from the 40,000-update online run. Overhead is 10.5% of ordinary training, dominated by probes and ghost inner products (91.3%). Boundaries take 120 seconds across 199 transitions, averaging 0.60 seconds (95th percentile: 0.89). Peak memory is 21.3 GiB/GPU; host controller state uses 3.8 MiB.

6.4 Application Study

We compare 160M seven-source checkpoints at 40,000 updates with Uniform, ADO, DoGraph, both DGA and R&B partitions, and the matched fixed cuts. Adaptation recipes are shared across methods, with the controller disabled. Figure 10 gives (a) equal-language code EM and (b)–(d) IMDb, AG News, and PAWS-Wiki accuracy; higher is better. S/16 identify source/16-group variants; Fix-S/Fix-16 denote matched controls. Bars and labels are means of three paired adaptation runs from one checkpoint per method; whiskers reflect adaptation, not pretraining-seed variability. Dotted lines mark Uniform. Truncated, differently scaled vertical axes require comparing numerical gains rather than bar heights across panels.

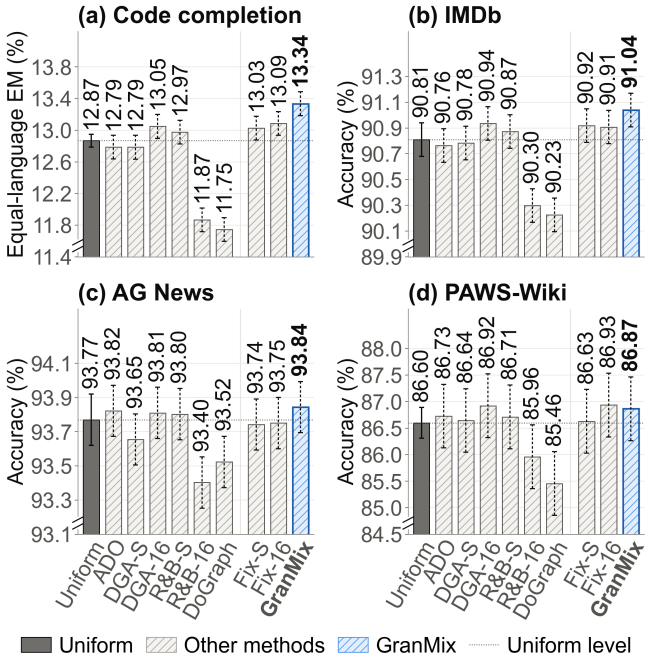


Figure 10: Downstream code-completion exact match and classification accuracy under shared adaptation recipes.

Code completion uses custom py150 [42] and CodeSearchNet Java [28] tests, with 50M adaptation tokens, a 50:50 language mixture, and learning rate 8×10^{-4}. Equal-language EM weights Python and Java equally. GranMix reaches 13.34%, versus Uniform's 12.87 ± 0.08%, gaining 0.47 and 0.25 points over Uniform and Fixed-16.

For IMDb [34], AG News [55], and PAWS-Wiki [56], we adapt all parameters with batch size 64 for three, three, and five epochs and input limits of 512, 256, and 256 tokens, respectively. In task order, backbone learning rates are 10^{-4}, 2×10^{-5}, and 5×10^{-5}; head rates are fivefold larger. GranMix's accuracies of 91.04%, 93.84%, and 86.87% exceed Uniform by 0.23, 0.07, and 0.27 points. It gains 0.13 and 0.09 points over Fixed-16 on IMDb and AG News, but trails Fixed-16 (86.93%) and DGA/semantic16 (86.92%) on PAWS-Wiki.

The 0.02-point AG News gain over ADO does not establish statistical significance across pretraining runs.

7 Related Works

We relate GranMix to database configuration, online adaptation, and workload-aware data organization, distinguishing sampling control from query-processing objectives.

7.1 Mixture Weight Optimization

Physical design selects indexes, materialized views, and partitions for a workload [2, 4, 10]. Parallel design advisors also choose data distribution and replication [37, 41]. OtterTune [48] learns configurations from past observations. Ding et al. [16] use query executions to improve index recommendations, addressing errors in estimated benefit. DoReMi [50] learns robust proxy mixtures, and DoGE [18] estimates generalization through bilevel optimization. RegMix and BiMix model mixture performance [21, 32]. Data-Juicer [11] supports recipe exploration. Unlike query-cost optimization, GranMix jointly selects regional sampling granularity and weights using a conservative objective and a shared group budget.

7.2 Online Data Mixing

Online physical design balances workload performance and reconfiguration cost [3, 7]. WFIT [43] uses work functions to account for index-transition costs. DBA Bandits [39] learns index utility from execution feedback; QueryBot 5000 [33] forecasts arrival rates, and Sibyl [26] predicts future query statements. In language-model training, ODM [5], DGA [17], ADO [30], and Aioli [12] adjust mixtures through bandit feedback, gradient alignment, learning potential, and online mixing laws, respectively. GREATS [49] uses validation-based contribution signals for online data selection. GranMix forecasts contributions across persistent candidate groups rather than incoming queries; calibrated intervals and handoff constrain updates while retaining candidate histories across changes in the active partition between control windows.

7.3 Grouping and Granularity in Data Mixing

STHoles [8] allocates synopsis resolution through query feedback; cracking [29] and stochastic cracking [25] reorganize queried regions. Qd-tree [52] builds a routing tree to organize records into workload-aware blocks; E-Store [46] and Clay [44] adapt fine-grained partitions to transaction load. These techniques target query access or distributed execution, whereas GranMix controls training exposure. CLIMB [15] and R&B [20] form semantic groups; DoGraph [51] revises gradient-based groups; HERMES [40] provides labels at several resolutions, and Olmix [13] revises mixtures as domains change. GranMix jointly optimizes a mixed-resolution cut of a fixed hierarchy and its sampling weights under a shared group budget. Inherited preferences connect successive cuts; updates require positive conservative gain after enforcing coverage and distribution-change constraints on the resulting sample distribution, rather than checking changes in group weights alone.

8 Conclusion

In this paper, we study adaptive partition granularity for online data mixing in language-model pretraining. By allowing coarse and fine groups to coexist across corpus regions, GranMix seeks to match sampling resolution to evolving training contributions rather than imposing a single granularity on the entire corpus. To jointly select data groups and sampling weights under a limited group budget, we: (1) formalize the partition-control task and characterize partition-induced control regret; (2) develop an online diagnosis algorithm that forecasts next-window contributions and calibrates prediction uncertainty; (3) design a budgeted tree-cut algorithm for joint partition and target-weight selection under a conservative control objective; (4) develop an executable handoff mechanism that preserves sampling coverage and limits distribution changes. Experiments on five SlimPajama subset workloads and the full seven-source corpus show improvements over uniform and common mixing methods, while paired comparisons with fixed-cut controls support the benefit of regional granularity adaptation.

References

[1] [n. d.]. . https://anonymous.4open.science/r/GranMix_source-5826/.

[2] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R Narasayya. 2000. Automated selection of materialized views and indexes in SQL databases. In *VLDB*, Vol. 2000. 496–505.

[3] Sanjay Agrawal, Eric Chu, and Vivek Narasayya. 2006. Automatic physical design tuning: workload as a sequence. In *SIGMOD*. 683–694.

[4] Sanjay Agrawal, Vivek Narasayya, and Beverly Yang. 2004. Integrating vertical and horizontal partitioning into automated physical database design. In *SIGMOD*. 359–370.

[5] Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. 2023. Efficient online data mixing for language model pre-training. *arXiv preprint arXiv:2312.02406* (2023).

[6] Maximilian Böther, Xiaozhe Yao, Tolga Kerimoglu, Dan Graur, Viktor Gsteiger, and Ana Klimovic. 2026. Mixtera: A Data Plane for Foundation Model Training. *PACMMOD (SIGMOD)* 4, 1, Article 54 (2026). doi:10.1145/3786668

[7] Nicolas Bruno and Surajit Chaudhuri. 2006. An online approach to physical design tuning. In *ICDE*. IEEE, 826–835.

[8] Nicolas Bruno, Surajit Chaudhuri, and Luis Gravano. 2001. STHoles: A Multidimensional Workload-Aware Histogram. In *SIGMOD*. 211–222. https://www.microsoft.com/en-us/research/publication/stholes-a-multidimensional-workload-aware-histogram/

[9] Chengliang Chai, Jiabin Liu, Nan Tang, Ju Fan, Dongjing Miao, Jiayi Wang, Yuyu Luo, and Guoliang Li. 2023. GoodCore: Data-effective and Data-efficient Machine Learning through Coreset Selection over Incomplete Data. *PACMMOD (SIGMOD)* 1, 2, Article 157 (2023), 27 pages. doi:10.1145/3589302

[10] Surajit Chaudhuri and Vivek R Narasayya. 1997. An efficient, cost-driven index selection tool for Microsoft SQL server. In *VLDB*, Vol. 97. San Francisco, 146–155.

[11] Daoyuan Chen, Yilun Huang, Zhijian Ma, Hesen Chen, Xuchen Pan, Ce Ge, Dawei Gao, Yuexiang Xie, Zhaoyang Liu, Jinyang Gao, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. Data-Juicer: A One-Stop Data Processing System for Large Language Models. In *SIGMOD Companion*. 120–134. doi:10.1145/3626246.3653385

[12] Mayee Chen, Michael Hu, Nicholas Lourie, Kyunghyun Cho, and Christopher Ré. 2025. Aioli: A unified optimization framework for language model data mixing. In *International Conference on Learning Representations*, Vol. 2025. 46089–46132.

[13] Mayee F. Chen, Tyler Murray, David Heineman, Matt Jordan, Hannaneh Hajishirzi, Christopher Ré, Luca Soldaini, and Kyle Lo. 2026. Olmix: A Framework for Data Mixing Throughout LM Development. *arXiv preprint arXiv:2602.12237* (2026). doi:10.48550/arXiv.2602.12237

[14] Ronald R. Coifman and Mladen Victor Wickerhauser. 1992. Entropy-Based Algorithms for Best Basis Selection. *IEEE Transactions on Information Theory* 38, 2 (1992), 713–718. doi:10.1109/18.119732

[15] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, et al. 2026. Nemotron-climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *Advances in Neural Information Processing Systems* 38 (2026).

[16] Bailu Ding, Sudipto Das, Ryan Marcus, Wentao Wu, Surajit Chaudhuri, and Vivek R. Narasayya. 2019. AI Meets AI: Leveraging Query Executions to Improve Index Recommendations. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 1241–1258. doi:10.1145/3299869.3324957

[17] Simin Fan, David Grangier, and Pierre Ablin. 2024. Dynamic gradient alignment for online data mixing. *arXiv preprint arXiv:2410.02498* (2024).

[18] Simin Fan, Matteo Pagliardini, and Martin Jaggi. 2023. Doge: Domain reweighting with generalization estimation. *arXiv preprint arXiv:2310.15393* (2023).

[19] David A. Freedman. 1975. On Tail Probabilities for Martingales. *The Annals of Probability* 3, 1 (1975), 100–118. doi:10.1214/aop/1176996452

[20] Albert Ge, Tzu-Heng Huang, John Cooper, Avi Trost, Ziyi Chu, Satya Sai Srinath Namburi GNVV, Ziyang Cai, Kendall Park, Nicholas Roberts, and Frederic Sala. 2025. R&B: Domain Regrouping and Data Mixture Balancing for Efficient Foundation Model Training. arXiv:2505.00358 [cs.LG] https://arxiv.org/abs/2505.00358

[21] Ce Ge, Zhijian Ma, Daoyuan Chen, Yaliang Li, and Bolin Ding. 2024. BiMix: A Bivariate Data Mixing Law for Language Model Pretraining. *arXiv preprint arXiv:2405.14908* (2024). doi:10.48550/arXiv.2405.14908

[22] Isaac Gibbs and Emmanuel J. Candès. 2021. Adaptive Conformal Inference Under Distribution Shift. In *Advances in Neural Information Processing Systems*, Vol. 34. 1660–1672. https://proceedings.neurips.cc/paper/2021/hash/0d441de75945e5acbc865406fc9a2559-Abstract.html

[23] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[24] Michael Günther, Jackmin Ong, Isabelle Mohr, Alaeddine Abdessalem, Tanguy Abel, Mohammad Kalim Akram, Susana Guzman, Georgios Mastrapas, Saba Sturua, Bo Wang, Maximilian Werk, Nan Wang, and Han Xiao. 2023. Jina Embeddings 2: 8192-Token General-Purpose Text Embeddings for Long Documents. arXiv:2310.19923 [cs.CL] https://arxiv.org/abs/2310.19923

[25] Felix Halim, Stratos Idreos, Panagiotis Karras, and Roland H. C. Yap. 2012. Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. *PVLDB* 5, 6 (2012), 502–513. doi:10.14778/2168651.2168652

[26] Hanxian Huang, Tarique Siddiqui, Rana Alotaibi, Carlo Curino, Jyoti Leeka, Alekh Jindal, Jishen Zhao, Jesús Camacho-Rodríguez, and Yuanyuan Tian. 2024. Sibyl: Forecasting Time-Evolving Query Workloads. *PACMMOD (SIGMOD)* 2, 1, Article 53 (2024), 27 pages. doi:10.1145/3639308

[27] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[28] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436 [cs.LG] doi:10.48550/arXiv.1909.09436

[29] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. 2007. Database Cracking. In *CIDR*. 68–78. https://stratos.seas.harvard.edu/publications/database-cracking

[30] Yiding Jiang, Allan Zhou, Zhili Feng, Sadhika Malladi, and Zico Kolter. 2025. Adaptive data optimization: Dynamic sample selection with scaling laws. In *International Conference on Learning Representations*, Vol. 2025. 58014–58037.

[31] Xiaotian Lin, Yanlin Qi, Yizhang Zhu, Themis Palpanas, Chengliang Chai, Nan Tang, and Yuyu Luo. 2025. LEAD: Iterative Data Selection for Efficient LLM Instruction Tuning. *PVLDB* 19, 3 (2025), 426–439. doi:10.14778/3778092.3778103

[32] Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. 2025. Regmix: Data mixture as regression for language model pre-training. In *International Conference on Learning Representations*, Vol. 2025. 38305–38339.

[33] Lin Ma, Dana Van Aken, Ahmed Hefny, Gustavo Mezerhane, Andrew Pavlo, and Geoffrey J. Gordon. 2018. Query-Based Workload Forecasting for Self-Driving Database Management Systems. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 631–645. doi:10.1145/3183713.3196908

[34] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning Word Vectors for Sentiment Analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Portland, Oregon, USA, 142–150. https://aclanthology.org/P11-1015/

[35] Iman Mirzadeh, Keivan Alizadeh-Vahid, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. 2025. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations*, Vol. 2025. 94743–94765.

[36] Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A Rossi, Seunghyun Yoon, and Hinrich Schütze. 2025. Nolima: Long-context evaluation beyond literal matching. *arXiv preprint arXiv:2502.05167* (2025).

[37] Rimma Nehme and Nicolas Bruno. 2011. Automated Partitioning Design in Parallel Database Systems. In *SIGMOD*. Association for Computing Machinery, New York, NY, USA, 1137–1148. doi:10.1145/1989323.1989444

[38] OpenAI. 2024. OpenAI o1-mini. https://openai.com/index/openai-o1-mini-advancing-cost-efficient-reasoning/. Accessed: 2026-10-02.

[39] R. Malinga Perera, Bastian Oetomo, Benjamin I. P. Rubinstein, and Renata Borovica-Gajic. 2021. DBA Bandits: Self-Driving Index Tuning under Ad-Hoc, Analytical Workloads with Safety Guarantees. In *ICDE*. IEEE, 600–611. doi:10.1109/ICDE51399.2021.00058

[40] Ziyun Qiao, Yue Min, Ruining Chen, and Yujun Li. 2026. HERMES: A Multi-Granularity Labeling Substrate for Pre-training Data Mixtures. *arXiv preprint arXiv:2607.02266* (2026).

[41] Jun Rao, Chun Zhang, Nimrod Megiddo, and Guy Lohman. 2002. Automating physical database design in a parallel database. In *SIGMOD*. 558–569.

[42] Veselin Raychev, Pavol Bielik, and Martin Vechev. 2016. Probabilistic Model for Code with Decision Trees. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '16)*. Association for Computing Machinery, 731–747. doi:10.1145/2983990.2984041

[43] Karl Schnaitter and Neoklis Polyzotis. 2012. Semi-Automatic Index Tuning: Keeping DBAs in the Loop. *PVLDB* 5, 5 (2012), 478–489. doi:10.14778/2140436.2140444

[44] Marco Serafini, Rebecca Taft, Aaron J. Elmore, Andrew Pavlo, Ashraf Aboulnaga, and Michael Stonebraker. 2016. Clay: Fine-Grained Adaptive Partitioning for General Database Schemas. *PVLDB* 10, 4 (2016), 445–456. doi:10.14778/3025111.3025125

[45] Chaoyuan Shen, Chi Zhang, Chengliang Chai, Jiacheng Wang, Jia Yuan, Yuping Wang, Ye Yuan, Guoren Wang, and Lei Cao. 2026. BRIEF: Bi-level Coreset Selection for Efficient Instruction Tuning in LLMs. *PVLDB* 19, 6 (2026), 1264–1277. doi:10.14778/3797919.3797933

[46] Rebecca Taft, Essam Mansour, Marco Serafini, Jennie Duggan, Aaron J Elmore, Ashraf Aboulnaga, Andrew Pavlo, and Michael Stonebraker. 2014. E-store: Fine-grained elastic partitioning for distributed transaction processing systems. *PVLDB* 8, 3 (2014), 245–256.

[47] Anvith Thudi, Evianne Rovers, Yangjun Ruan, Tristan Thrush, and Chris J Maddison. 2025. Mixmin: Finding data mixtures via convex minimization. *arXiv preprint arXiv:2502.10510* (2025).

[48] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *SIGMOD*. 1009–1024. doi:10.1145/3035918.3064029

[49] Jiachen T Wang, Tong Wu, Dawn Song, Prateek Mittal, and Ruoxi Jia. 2024. Greats: Online selection of high-quality data for llm training in every iteration. *Advances in Neural Information Processing Systems* 37 (2024), 131197–131223.

[50] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy S Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. 2023. Doremi: Optimizing data mixtures speeds up language model pretraining. *Advances in Neural Information Processing Systems* 36 (2023), 69798–69818.

[51] Yuanjian Xu, Tianze Sun, Changwei Xu, XinLong Zhao, Jianing Hao, Ran Chen, Yang Liu, Ruijie Xu, Stephen Chen, and Guang Zhang. 2026. Rethinking Data Mixing from the Perspective of Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 337–349.

[52] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. 2020. Qd-tree: Learning data layouts for big data analytics. In *SIGMOD*. 193–208.

[53] Zhaojian Yu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. 2025. HumanEval pro and MBPP pro: Evaluating large language models on self-invoking code generation task. In *Findings of the Association for Computational Linguistics: ACL 2025*. 13253–13279.

[54] Chi Zhang, Jiacheng Wang, Kun He, Chengliang Chai, Yunpeng Zhang, Yuping Wang, Xu Zhou, Linan Zheng, Lijun Wu, Conghui He, and Lei Cao. 2026. Data-efficient Online Training for Direct Alignment in LLMs. *PVLDB* 19, 10 (2026), 2741–2754. doi:10.14778/3828612.3828628

[55] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *Advances in Neural Information Processing Systems*, Vol. 28. https://proceedings.neurips.cc/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html

[56] Yuan Zhang, Jason Baldridge, and Luheng He. 2019. PAWS: Paraphrase Adversaries from Word Scrambling. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 1298–1308. doi:10.18653/v1/N19-1131

A Proofs

A.1 Proof of Theorem 1

Fix a boundary and abbreviate q_t^- and v_t by q^- and v. The finite corpus, positive p, q^-, bounded contributions, and $\eta > 0$ ensure that every normalizing constant below is finite and positive. We first optimize at sample level, then identify the restriction imposed by a cut. Set $Z_{\text{sample}} = \sum_i q^-(i)e^{\eta v(i)}$ and normalize these tilted masses to obtain $q^\star$. Substituting its logarithm into the KL divergence gives

$$\begin{aligned} q^\star(i) &= \frac{q^-(i)e^{\eta v(i)}}{Z_{\text{sample}}}, \\ J_t(q) &= \eta^{-1}\log Z_{\text{sample}} - \eta^{-1}D_{\text{KL}}(q\|q^\star). \end{aligned} \tag{A.1}$$

Thus KL nonnegativity makes $q^\star$ the unique maximizer, with value $\eta^{-1}\log Z_{\text{sample}}$. The convention $0\log(0/a) = 0$ includes distributions on the boundary of the simplex.

On a fixed cut Π, the within-group distribution is prescribed by p. Averaging the log density ratio within each nonempty group gives

$$\begin{aligned} D_{\text{KL}}(q_{\Pi,w}\|q^-) = &\sum_{G\in\Pi} w_G \log\frac{w_G}{p(G)} \\ &- \sum_{G\in\Pi} w_G\kappa_t(G), \end{aligned} \tag{A.2}$$

where $\kappa_t(G) = \mathbb{E}_{p(\cdot|G)}[\log(q^-/p)]$. This separates the cost of allocating group mass from inherited preferences within a group. Continuity of $x\log x$ extends it to zero group weights. Define $a_G = p(G)e^{\eta v_t(G)+\kappa_t(G)}$, $Z_\Pi = \sum_G a_G$, and $\bar{w}_G = a_G/Z_\Pi$. The same normalization argument now gives

$$J_t(q_{\Pi,w}) = \eta^{-1}\log Z_\Pi - \eta^{-1}D_{\text{KL}}(w\|\bar{w}).$$

Hence $\bar{w}$ is uniquely optimal on the cut. Both optimizations use full probability simplexes; coverage floors and the hard KL budget are excluded from this representation comparison.

Using the theorem's control profile ϕ_t, the two normalizers are $\mathbb{E}_p[e^{\phi_t(i)}]$ and $\sum_G p(G)e^{\bar{\phi}_t(G)}$. Their optimal-value difference is therefore

$$\mathcal{R}_t(\Pi) = \eta^{-1}\log\frac{Z_{\text{sample}}}{Z_\Pi},$$

which is Equation (3). Jensen's inequality within every group makes the numerator at least the denominator. Since all reference masses are positive and the exponential is strictly convex, equality holds exactly when ϕ_t is constant in every group. If q^-/p is groupwise constant, only the contribution term can vary there, giving the stated equality condition.

For the small-η result, set $q^- = p$ and hold p, v, Π fixed. The sample contributions and their group means share mean μ. Write their variances as σ^2 and σ_Π^2, respectively. Bounded contributions give bounded third derivatives of both log moment-generating functions near zero, so

$$\begin{aligned} \log Z_{\text{sample}} &= \eta\mu + \tfrac{1}{2}\eta^2\sigma^2 + O(\eta^3), \\ \log Z_\Pi &= \eta\mu + \tfrac{1}{2}\eta^2\sigma_\Pi^2 + O(\eta^3). \end{aligned}$$

The common mean cancels, while total variance identifies $\sigma^2 - \sigma_\Pi^2 = V_t^{\text{within}}(\Pi)$, the reference-weighted within-group variance. Dividing the difference by η yields

$$\mathcal{R}_t(\Pi) = \frac{\eta}{2}V_t^{\text{within}}(\Pi) + O(\eta^2). \tag{A.3}$$

A.2 Proof of Proposition 1

Use each window's saved forecasts, base scales, and multiplier. To express the containment check as a scalar threshold, let $d_t^{\text{aud}}(G)$ be the greater distance from $\hat{v}_t(G)$ to the two audit endpoints, and set

$$S_t = \max_{G\in\mathcal{V}_t}\frac{d_t^{\text{aud}}(G)}{b_t(G)}, \qquad S_t = 0 \text{ if } \mathcal{V}_t = \varnothing.$$

The saved prediction interval is centered at $\hat{v}_t(G)$ with radius $c_t b_t(G)$; thus every audit interval is contained exactly when $S_t \le c_t$. In particular, $\text{err}_t = \mathbf{1}\{S_t > c_t\}$. Prediction centers and audit endpoints lie in $[-M, M]$, and $b_t(G) \ge b_{\min}$, so $S_t \le S_{\max} := 2M/b_{\min}$.

This bound prevents the calibration multiplier from growing indefinitely. At $c_t \ge S_{\max}$, every check passes and the update decreases c_t, subject to its nonnegative floor. Below $S_{\max}$, a single update increases it by less than γ_{cal}. Induction therefore gives $0 \le c_t \le C_{\max}$. Moreover, the lower projection can only increase the unprojected update, implying

$$\gamma_{\text{cal}}\sum_{t=1}^{T}(\text{err}_t - \zeta) \le c_{T+1} - c_1 \le C_{\max} - c_1.$$

On the simultaneous audit event $\mathcal{A}$ of Equation (B.1), containment transfers audit coverage to every saved prediction interval. Consequently $z_t \le \text{err}_t$, also for an empty valid set, and

$$\frac{1}{T}\sum_{t=1}^{T} z_t \le \zeta + \frac{C_{\max} - c_1}{\gamma_{\text{cal}}T}.$$

Because the same event covers all nodes and windows, this holds simultaneously for every T, with probability at least $1 - \delta_{\text{aud}}$. The argument is pathwise and requires chronological use of saved records; it needs neither stationarity nor independence of forecasts, node sets, or contributions.

A.3 Proof of Proposition 2

Fix a boundary with finite prediction intervals and suppress time indices. Both samplers retain the reference conditional distribution within their respective groups. Their expected contributions therefore equal their group-weighted means. Since $D_{\text{KL}}(q^-\|q^-) = 0$, their exact difference is

$$\begin{aligned} \Delta J(q_{\Pi,w}) = &\sum_{G\in\Pi} w_G v(G) - \sum_{B\in\Pi^-} q^-(B)v(B) \\ &- \eta^{-1}D_{\text{KL}}(q_{\Pi,w}\|q^-). \end{aligned}$$

Positivity of the installed sampler makes this finite, including zero candidate weights under $0\log(0/a) = 0$.

The excess error measures precisely how far a node contribution falls outside its forecast interval. Its definition gives

$$L(G) - e(G) \le v(G) \le U(G) + e(G).$$

To lower-bound improvement, apply the lower inequality to the candidate and the upper inequality to the installed sampler. All weights are nonnegative, so averaging preserves these inequalities. The candidate's loss below its lower estimate and the installed sampler's excess above its upper estimate sum to $E(\Pi, w)$. The KL cost is unchanged, yielding

$$\Delta J(q_{\Pi,w}) \ge C(\Pi, w) - E(\Pi, w).$$

Coverage of both cuts makes every excess error zero, so positive conservative gain ensures improvement. Without coverage, $C(\Pi, w) > E(\Pi, w)$ suffices. These comparisons hold pointwise for every candidate on the same realized contribution field, including one selected using the forecasts; no independence assumption is needed.

A.4 Proof of Theorem 2

Fix a valid cut Π. Positive reference and installed probabilities, nonempty groups, finite lower bounds, and $\eta > 0$ make each score $A(G) = p(G)e^{\eta L(G)+\kappa(G)}$ positive and finite. Thus $Z_\Pi = \sum_G A(G)$ normalizes them to the strictly positive probability vector $w^\star$. We optimize over the full group-weight simplex; handoff imposes coverage floors and the execution budget afterward.

Equation (A.2) expresses the sample-level intervention cost in terms of group weights. Substituting it into the conservative objective combines each lower contribution bound, reference mass, and inherited preference into $A(G)$:

$$C(\Pi, w) + U^- = \eta^{-1} \sum_{G\in\Pi} w_G \log \frac{A(G)}{w_G}.$$

Replacing $A(G)$ by $Z_\Pi w^\star_G$ and using unit total weight gives the variational identity

$$\begin{aligned} C(\Pi, w) = \eta^{-1} \log Z_\Pi - U^- \\ - \eta^{-1} D_{\mathrm{KL}}(w \| w^\star). \end{aligned} \tag{A.4}$$

The continuous extension of $x \log x$ at zero makes this valid on the entire simplex. KL nonnegativity shows that the normalized scores achieve the optimum, and equality holds only at $w = w^\star$. This uniqueness also excludes boundary alternatives because every coordinate of $w^\star$ is positive. Hence

$$C^\star(\Pi) = \eta^{-1} \log Z_\Pi - U^-,$$

as claimed.

Finally, let $\mathcal{C}$ be the nonempty finite family of valid cuts with at most m groups. Their node scores are fixed during search, and U^- is common to all candidates. Strict monotonicity of the logarithm therefore gives

$$\arg\max_{\Pi\in\mathcal{C}} C^\star(\Pi) = \arg\max_{\Pi\in\mathcal{C}} \sum_{G\in\Pi} A(G).$$

Thus the additive cut search jointly optimizes the cut and its target weights. Several cuts may tie, but each fixed cut has a unique optimal weight vector.

A.5 Proof of Theorem 3

Handoff must first account for the change imposed by the proposed partition, then determine how far the weights can move toward the mixer target. Fix Π^+, suppress time indices, and let $n = |\Pi^+|$. All sums below range over its groups. The assumed positivity of $p, q^-, \widetilde{w}$, together with $0 < \varepsilon < 1$, ensures positive inherited weights $r^0(G)$ and floors $\ell_G = \varepsilon p(G)$, whose sum is ε.

Substituting $\log r^0(G) = \log p(G) + \kappa(G) - \log Z$ into Equation (A.2) gives

$$K(w) = D_{\mathrm{KL}}(w \| r^0) - \log Z. \tag{A.5}$$

Jensen's inequality applied to $\log(q^-(i)/p(i))$ within each group gives $p(G)e^{\kappa(G)} \le q^-(G)$, hence $0 < Z \le 1$. Thus $-\log Z$ is a nonnegative representation cost: without floors, it is the minimum divergence, attained uniquely at r^0. Equality $Z = 1$ holds exactly when q^-/p is constant within every proposed group. In particular, a refinement of the installed cut has $Z = 1$ and $r^0(G) = q^-(G)$. This distinction matters when the partition merges regions with different installed preferences: even the closest representable sampler then uses part of the KL budget. Testing only the divergence between group weights would overlook that unavoidable cost and could accept a sampler whose sample-level change exceeds the budget.

Write $f(w) = D_{\mathrm{KL}}(w\|\widetilde{w})$, $g(w) = D_{\mathrm{KL}}(w\|r^0)$, and $\mathcal{W}_\ell = \{w : \sum_G w_G = 1,\ w_G \ge \ell_G\}$. This compact set is nonempty, with $(p(G))_G$ strictly above its floors. Both KL objectives have positive definite Hessian $\mathrm{diag}(1/w_G)$. Consequently, each $h_\alpha = \alpha f + (1-\alpha)g$, for $\alpha \in [0,1]$, has a unique minimizer. Set $z_G(\alpha) = \widetilde{w}_G^\alpha r^0(G)^{1-\alpha}$. The KKT conditions for h_α, with equality multiplier ν and nonnegative floor multipliers μ_G, yield

$$\begin{aligned} \log \frac{w_G}{z_G(\alpha)} + 1 + \nu - \mu_G = 0, \\ \mu_G(w_G - \ell_G) = 0, \\ w_G(\alpha) = \max\{\ell_G, z_G(\alpha)/Z_\alpha\}, \end{aligned} \tag{A.6}$$

where $Z_\alpha = e^{1+\nu}$. Free coordinates are proportional to z; a coordinate at its floor has $z_G/Z_\alpha \le \ell_G$. Conversely, these relations satisfy stationarity and complementary slackness, so the normalized vector is the minimizer. The parameter α therefore interpolates between two optimization objectives, while the floors preserve coverage throughout. Clipping alone would not preserve unit mass; the shared denominator redistributes the remaining mass among free coordinates.

The normalizer is unique. Indeed, $\Phi_\alpha(t) = \sum_G \max\{\ell_G, z_G(\alpha)/t\}$ is continuous, tends to infinity at zero, and tends to $\varepsilon < 1$ at infinity. At level one, at least one coordinate exceeds its floor, so increasing t strictly decreases the sum. The path is also continuous: any subsequential limit of minimizers as $\alpha_j \to \alpha$ minimizes h_α, by continuity and compactness, and uniqueness identifies that limit with $w(\alpha)$.

To establish the direction of travel, for $\alpha > 0$ write $\lambda = (1-\alpha)/\alpha$; the same vector minimizes $f + \lambda g$. For $\lambda_1 > \lambda_2$, let $\Delta f = f(w_{\lambda_1}) - f(w_{\lambda_2})$ and define Δg similarly. Optimality gives $\Delta f + \lambda_1 \Delta g \le 0$ and $\Delta f + \lambda_2 \Delta g \ge 0$. Hence $\Delta g \le 0$ and $\Delta f \ge 0$. Since λ decreases with α, continuity extends these conclusions to zero:

$$\begin{aligned} &K(w(\alpha_1)) \le K(w(\alpha_2)), \\ &f(w(\alpha_1)) \ge f(w(\alpha_2)) \qquad (\alpha_1 \le \alpha_2). \end{aligned} \tag{A.7}$$

Thus moving along the path improves agreement with the mixer while consuming the change budget.

The endpoint $w(0)$ uniquely minimizes K over $\mathcal{W}_\ell$, proving feasibility exactly when $K(w(0)) \le \delta$. When feasible, the executable set $\mathcal{E}$ is compact and convex, and f has a unique minimizer there. Continuity and monotonicity make the feasible path parameters a closed interval $[0, \alpha^\star]$. If $w(1)$ is feasible, it is already optimal. If $\delta = K(w(0)) < K(w(1))$, strict convexity makes $w(0)$ the only feasible vector. The path may remain at this vector over an initial interval, so $\alpha^\star$ need not equal zero. Otherwise, $0 < \alpha^\star < 1$ and $K(w(\alpha^\star)) = \delta$. Put $w^\dagger = w(\alpha^\star)$ and $\lambda^\star = (1-\alpha^\star)/\alpha^\star$. For every

feasible v, optimality of $f + \lambda^\star g$ gives

$$f(w^\dagger) \le f(v) + \lambda^\star \left[g(v) - g(w^\dagger)\right] \le f(v). \tag{A.8}$$

These cases distinguish a target that is already executable, a budget exhausted by the minimum-change sampler, and an active budget that stops the path between the endpoints. They also show why strict monotonicity is unnecessary: a constant portion of the path does not create multiple optimal weight vectors. This proves the claimed constrained optimum in every case. The floors also give $q_w(i) \ge \varepsilon p(i)$, and therefore importance weights at most ε^{-1}.

For completeness, FloorNormalize computes this path exactly. Let S contain the fixed coordinates, R the free ones, $M_S = 1 - \sum_{G\in S} \ell_G > 0$, and $Z_S = \sum_{G\in R} z_G/M_S$. For the nonempty set B of free coordinates below their floors, write $u = \sum_{G\in B} z_G$ and $b = \sum_{G\in B} \ell_G$. Then $u < Z_S b$, and fixing these coordinates increases the denominator:

$$Z_{S\cup B} - Z_S = \frac{Z_S b - u}{M_S - b} > 0. \tag{A.9}$$

Previously fixed coordinates consequently remain below their floors before clipping. A free coordinate always remains: their provisional mass is M_S, which exceeds the sum of their floors because $\varepsilon < 1$. Each nonterminal pass fixes at least one new coordinate, so there are at most $n - 1$ such passes. At termination, all free coordinates satisfy their floors, and the constructed vector has unit mass and the maximum form in Equation (A.6). Uniqueness of the normalizer completes the argument. The increasing-denominator invariant is essential here: once a coordinate is fixed, later passes cannot make it eligible for release. Consequently the procedure needs no reversals or enumeration of possible active sets.

Algorithm 3 handles infeasibility and both feasible endpoint cases exactly. In the remaining case, midpoint tests preserve

$$\begin{aligned} &K(w(a_s)) \le \delta < K(w(b_s)), \\ &a_s \le \alpha^\star < b_s, \qquad b_s - a_s = 2^{-s}. \end{aligned} \tag{A.10}$$

After at most $\lceil \log_2(1/\xi) \rceil$ iterations, returning $w(a_s)$ is feasible and gives $0 \le \alpha^\star - a_s \le \xi$. Returning the lower endpoint preserves the KL constraint at every stopping tolerance, even when the optimal parameter lies between the last two tests. This is a parameter tolerance; finite bisection need not return the exact optimizer. Continuity ensures convergence of both weights and objective as $\xi \downarrow 0$. Direct normalization scans cost $O(n^2)$, and each KL evaluation costs $O(n)$, giving total time $O(n^2[1 + \log(1/\xi)])$ and space $O(n)$. Evaluate z from its logs after subtracting a common offset, which leaves normalization unchanged. Feasibility holds in exact arithmetic; finite-precision output is checked for unit mass, floors, and the full budget $g(w) - \log Z \le \delta$.

A.6 Proof of Corollary 1

The relevant loss compares the returned weights with the original conservative target $w^\star$, so it includes both the mixer adjustment and the execution constraints. Equation (A.4) and Proposition 2 give, for any feasible proposal,

$$\begin{aligned} R^{\text{exec}} &= C^\star(\Pi^+) - C(\Pi^+, w^{\text{exec}}) \\ &= \eta^{-1} D_{\text{KL}}(w^{\text{exec}} \| w^\star) \ge 0, \\ \Delta J(q^{\text{exec}}) &\ge C^\star(\Pi^+) - R^{\text{exec}} \\ &\quad - E(\Pi^+, w^{\text{exec}}). \end{aligned} \tag{A.11}$$

For the average downside, let z_t indicate any prediction miss among valid nodes. Since $v_t(G), \hat{v}_t(G) \in [-M, M]$ and $\beta_t(G) \ge 0$, the excess error $e_t(G) = (|v_t(G) - \hat{v}_t(G)| - \beta_t(G))_+$ satisfies $0 \le e_t(G) \le 2Mz_t$. An installed update uses valid current and proposed cuts; their respective weights each sum to one. The two weighted error contributions therefore give $E_t \le 4Mz_t$. Algorithm 2 accepts only positive conservative gain, so its downside is at most E_t. On Keep, including an invalid current cut, the sampler remains q_t^- and the gain is zero. Thus $[-\Delta J_t(q_t^{\text{exec}})]_+ \le 4Mz_t$ in every window.

On the simultaneous audit event in Equation (B.1), Proposition 1 bounds the frequency of these misses. Summing the preceding window bound yields

$$\begin{aligned} \frac{1}{T}\sum_{t=1}^{T} [-\Delta J_t(q_t^{\text{exec}})]_+ &\le \frac{4M}{T}\sum_{t=1}^{T} z_t \\ &\le 4M\left(\zeta + \frac{C_{\max} - c_1}{\gamma_{\text{cal}} T}\right). \end{aligned} \tag{A.12}$$

The bound concerns the window objective used by the controller. Its interpretation is that negative executed gain can occur only on windows with a prediction miss, and bounded contributions limit the possible loss on each such window. The event has probability at least $1 - \delta_{\text{aud}}$ and holds simultaneously for all $T \ge 1$, establishing Equation (16).

B Additional Analysis

B.1 Local Taylor approximation and window-gain transfer

Fix the pre-update state θ_s, example i, domain probe $g^V_{s,d(i)} = \nabla L_{V,d(i)}(\theta_s)$, and step size $h_s > 0$. Define $\Delta_s(i) = L_{V,d(i)}(\theta_s) - L_{V,d(i)}(\theta_s - h_s g_s(i))$. If the validation gradient is $L_{d(i)}$-Lipschitz along this segment, integrating it shows that the first-order prediction $h_s\langle g_s(i), g^V_{s,d(i)}\rangle$ differs from $\Delta_s(i)$ by at most $L_{d(i)} h_s^2 \|g_s(i)\|^2/2$. Each domain uses its own objective and positive scale $a^V_{s,d(i)}$, held fixed at the pre-update state. This is the single-example gradient-descent scoring step underlying GREATS [49].

Let $u_s^{\text{loc}}(i) = \text{clip}_M(\Delta_s(i)/(h_s a^V_{s,d(i)}))$, and let $u_s(i)$ be the normalized, clipped inner product in Section 3.1. Since clipping is 1-Lipschitz and both scores lie in $[-M, M]$,

$$\begin{aligned} &|u_s^{\text{loc}}(i) - u_s(i)| \le \epsilon_s(i), \\ &\epsilon_s(i) = \min\left\{2M, \frac{L_{d(i)} h_s \|g_s(i)\|^2}{2a^V_{s,d(i)}}\right\}. \end{aligned}$$

Average over the H steps in $\mathcal{W}_t$ to obtain $v_t^{\text{loc}}(i) = H^{-1}\sum_s u_s^{\text{loc}}(i)$ and $\rho_t(i) = H^{-1}\sum_s \epsilon_s(i)$. Thus $\delta_t(i) = v_t^{\text{loc}}(i) - v_t(i)$ satisfies $|\delta_t(i)| \le \rho_t(i)$. Define J_t^{loc} by replacing v_t in Equation (1) while retaining its KL penalty. The two gain comparisons differ by $\sum_i (q(i) -$

$q_t^-(i))\delta_t(i)$, whose absolute value is at most $\sum_i |q(i) - q_t^-(i)|\rho_t(i)$. In particular,

$$\begin{aligned} J_t^{\mathrm{loc}}(q) - J_t^{\mathrm{loc}}(q_t^-) \\ \geq J_t(q) - J_t(q_t^-) \\ - \sum_i \big(q(i) + q_t^-(i)\big)\rho_t(i). \end{aligned}$$

The correction is weighted by the change in sampling exposure: examples whose probabilities do not change contribute no transfer error. This transfers the comparison to finite-step gains at the same pre-update states; it does not compare trajectories generated by different samplers.

B.2 Lossless grouping under separable regularization

On a finite corpus, assume $p(i), q^-(i), \eta > 0$. Let $f : [0,\infty) \to \mathbb{R} \cup \{+\infty\}$ be lower semicontinuous, strictly convex on its effective domain, and finite and continuously differentiable on $(0,\infty)$. Allow either finite $f(0)$ or $f(0) = +\infty$, and apply this same cost to every relative intervention:

$$\begin{aligned} J_f(q) = \sum_i q(i)v(i) \\ - \eta^{-1} \sum_i q^-(i) f\left(\frac{q(i)}{q^-(i)}\right). \end{aligned}$$

Assume the maximum is attained at a unique interior distribution $q_f^\star$. Group samplers retain $p(i \mid G)$. The relevant question is whether tying those examples to one weight excludes the unconstrained optimum. This distinguishes loss caused by the partition from loss caused by the particular optimization procedure.

Proposition B.1 (Lossless grouping with separable costs)**.** *A partition has zero loss under J_f exactly when $q_f^\star(i)/p(i)$ is constant within every group. If q^-/p is also constant within groups, this is equivalent to equal contributions $v(i)$ within each group; $q^- = p$ is a special case.*

Write $x_i = q(i)/q^-(i)$ and $x_i^\star = q_f^\star(i)/q^-(i) > 0$. Interior stationarity gives $f'(x_i^\star) = \eta(v(i) - \lambda)$ for one multiplier λ. For $a \geq 0, b > 0$, define $D_f(a,b) = f(a) - f(b) - f'(b)(a-b)$. Normalization cancels the multiplier, yielding

$$J_f(q_f^\star) - J_f(q) = \eta^{-1} \sum_i q^-(i) D_f(x_i, x_i^\star).$$

Strict convexity makes every summand nonnegative, with equality only at $x_i = x_i^\star$. This also covers zero coordinates when $f(0)$ is finite; otherwise such coordinates give $J_f(q) = -\infty$.

The minimum gap over the closed group-weight simplex is attained: that simplex is compact, the sampler map is continuous, the gap is lower semicontinuous, and $w_G = p(G)$ gives the finite-value sampler p. Zero loss therefore requires representability of $q_f^\star$, equivalently a constant ratio $c_G = q_f^\star(i)/p(i)$, represented by $w_G = c_G p(G)$. If $q^-(i)/p(i) = b_G > 0$, this ratio equals $b_G x_i^\star$. Strictly increasing f' and stationarity make its constancy equivalent to equal $v(i)$.

For KL, $f(x) = x\log x - x + 1$, with $f(0) = 1$. Stationarity gives $q_f^\star(i) = q^-(i)e^{\eta v(i)}/Z_{\mathrm{sample}}$, hence $q_f^\star(i)/p(i) = e^{\phi_t(i)}/Z_{\mathrm{sample}}$. This recovers Theorem 1 and the exponential scores in Section 4.

B.3 Weighted history moments and forecast features

Let τ divide H. Retain up to $K_{\mathrm{hist}} \geq H/\tau$ completed blocks in $\mathcal{B}_t$, keeping completed-window blocks until their audit finishes. For nonempty history, set $b_\star = \max \mathcal{B}_t$ and $h_{t,b} = \lambda_{\mathrm{hist}}^{b_\star - b}$, with $0 < \lambda_{\mathrm{hist}} \leq 1$. The stored totals produce

$$\begin{aligned} W_t(G) &= \sum_{b\in\mathcal{B}_t} h_{t,b} W_{b,G}, \\ W_{2,t}(G) &= \sum_{b\in\mathcal{B}_t} h_{t,b}^2 W_{2,b,G}, \\ T_{1,t}(G) &= \sum_{b\in\mathcal{B}_t} h_{t,b} S_{b,G}/a_b, \\ T_{2,t}(G) &= \sum_{b\in\mathcal{B}_t} h_{t,b} Q_{b,G}/a_b^2. \end{aligned}$$

For $W_t(G) > 0$, the weighted population moments are

$$\begin{aligned} \bar{z}_t(G) &= T_{1,t}(G)/W_t(G), \\ s_t^2(G) &= \max\{0, T_{2,t}(G)/W_t(G) - \bar{z}_t(G)^2\}, \\ n_{\mathrm{eff},t}(G) &= W_t(G)^2/W_{2,t}(G). \end{aligned}$$

An observation's weight is $h_{t,b}\omega$; squaring it explains the squared decay in $W_{2,t}$. When $W_t(G) = 0$, including initialization, set the effective count to zero, skip all divisions, and return $(\bot, \bot, 0, \bot)$; the node is invalid.

For nonempty node blocks $\mathcal{B}_t^+(G) = \{b \in \mathcal{B}_t : W_{b,G} > 0\}$, define

$$\begin{aligned} \mu_{b,G} &= S_{b,G}/(a_b W_{b,G}), \\ \pi_{t,b,G} &= h_{t,b} W_{b,G}/W_t(G), \\ s_t^{\mathrm{blk}}(G)^2 &= \sum_{b\in\mathcal{B}_t^+(G)} \pi_{t,b,G}(\mu_{b,G} - \bar{z}_t(G))^2. \end{aligned}$$

These block weights sum to one and reproduce $\bar{z}_t(G)$. Restrict the sums defining $W_t, T_{1,t}$ to the latest up to $1 \leq K_{\mathrm{recent}} \leq K_{\mathrm{hist}}$ retained blocks to define $W_t^{\mathrm{recent}}, T_{1,t}^{\mathrm{recent}}$. Their ratio is $\bar{z}_t^{\mathrm{recent}}$; when its denominator is zero, use $\bar{z}_t$. Set $d_t(G) = |\bar{z}_t^{\mathrm{recent}}(G) - \bar{z}_t(G)|$, and use the latest nonempty node block $b_G = \max \mathcal{B}_t^+(G)$ for $m_t(G) = \mu_{b_G,G}$.

The root scale is $a_b = \max\{a_{\min}, \sqrt{Q_{b,r}/W_{b,r}}\}$, with $a_{\min} > 0$, and the latest global block supplies $a_t = a_{b_\star}$. Thus

$$\begin{aligned} \hat{v}_t(G) &= \mathrm{clip}_M(a_t \bar{z}_t(G)), \\ b_t(G) &= b_{\min} + a_t\Bigg(\lambda_1 \frac{s_t(G)}{\sqrt{n_{\mathrm{eff},t}(G)}} \\ &\qquad + \lambda_2 s_t^{\mathrm{blk}}(G) + \lambda_3 d_t(G)\Bigg). \end{aligned}$$

Here $b_{\min} > 0$ and $\lambda_j \geq 0$. History settings and feature coefficients are fixed before training; only the width multiplier updates. Finite outputs and $n_{\mathrm{eff},t}(G) \geq n_{\min} > 0$ determine validity. Saved forecasts and base scales remain unchanged until audit. The three features capture distinct sources of uncertainty: dispersed observations with limited effective evidence, changes between historical blocks, and disagreement between recent and long histories. Missing recent evidence therefore yields zero drift.

B.4 Audit coverage bound

Fix window $t \geq 1$, node G with $p(G) > 0$, and $N = HB$ draws. The pre-draw history $\mathcal{F}_{t,j-1}$ includes the model state and domain probes. Require $u_{t,j}(\cdot), q_{t,j}(\cdot)$ to be measurable with respect to it, and $\mathbb{P}(I_{t,j} = i \mid \mathcal{F}_{t,j-1}) = q_{t,j}(i)$. Assume $|u_{t,j}(i)| \leq M$ and $q_{t,j}(i) \geq \varepsilon p(i)$, with $0 < \varepsilon \leq 1$. These conditional assumptions permit adaptive training and dependent draws, as in martingale off-policy inference.[1]

Unlike the self-normalized historical forecast, the audit uses a fixed denominator. It measures the completed window directly, before block-scale normalization, so its conditional expectation matches the reference contribution being audited. Importance weighting recovers that conditional reference mean:

$$\begin{aligned} X_{t,j,G} &= \frac{\mathbf{1}\{I_{t,j} \in G\} p(I_{t,j})}{p(G) q_{t,j}(I_{t,j})} u_{t,j}(I_{t,j}), \\ \mu_{t,j,G} &= \mathbb{E}[X_{t,j,G} \mid \mathcal{F}_{t,j-1}] \\ &= \sum_{i \in G} p(i \mid G) u_{t,j}(i). \end{aligned}$$

Each step's B draws share its pre-update score, so $N^{-1} \sum_j \mu_{t,j,G} = v_t(G)$. Consequently, $D_{t,j,G} = X_{t,j,G} - \mu_{t,j,G}$ is a martingale difference and $\overline{X}_{t,G} = S^{\mathrm{win}}_{t,G}/(Np(G))$ has error $N^{-1} \sum_j D_{t,j,G}$.

The coverage floor bounds the conditional second moment by $M^2/(\varepsilon p(G))$. Also $|\mu_{t,j,G}| \leq M$ and $|X_{t,j,G}| \leq M/(\varepsilon p(G))$. Thus the increment bound is $b^{\mathrm{aud}}_G = 2M/(\varepsilon p(G))$, and the summed conditional variance is at most $V^{\mathrm{aud}}_G = NM^2/(\varepsilon p(G))$. Bernstein–Freedman [19] gives, for $a > 0$,

$$\mathbb{P}\left(\left|\sum_j D_{t,j,G}\right| \geq a\right) \leq 2 \exp\left\{-\frac{a^2}{2(V^{\mathrm{aud}}_G + b^{\mathrm{aud}}_G a/3)}\right\}.$$

Allocate $\delta_t = \delta_{\mathrm{aud}}/[t(t+1)]$, and set $\chi_t = \log(2|\mathcal{T}|/\delta_t)$. Solving the exponent equation for its positive root gives

$$\begin{aligned} x_t(G) &= \frac{\chi_t}{N\varepsilon p(G)}, \\ R^{\mathrm{aud}}_t(G) &= M\left(\frac{2x_t(G)}{3} + \sqrt{2x_t(G) + \frac{4x_t(G)^2}{9}}\right), \\ r^{\mathrm{aud}}_t(G) &= \min\{2M, R^{\mathrm{aud}}_t(G)\}. \end{aligned}$$

The untruncated radius bounds the error of $\overline{X}_{t,G}$ except with probability $\delta_t/|\mathcal{T}|$. Projection gives $\breve{v}_t(G) = \mathrm{clip}_M(\overline{X}_{t,G})$ and bounds its error by the smaller of $2M$ and the unprojected error. Hence the clipped endpoints $\check{L}_t(G) = \max\{-M, \breve{v}_t(G) - r^{\mathrm{aud}}_t(G)\}$ and $\check{U}_t(G) = \min\{M, \breve{v}_t(G) + r^{\mathrm{aud}}_t(G)\}$ retain coverage; radius $2M$ yields all of $[-M, M]$.

Summing failure probabilities over the fixed tree and all windows uses $\sum_{t \geq 1} 1/[t(t+1)] = 1$. Therefore

$$\begin{aligned} \mathcal{A} &= \bigcap_{t \geq 1} \bigcap_{G \in \mathcal{T}} \{v_t(G) \in [\check{L}_t(G), \check{U}_t(G)]\}, \\ \mathbb{P}(\mathcal{A}) &\geq 1 - \delta_{\mathrm{aud}}. \end{aligned} \tag{B.1}$$

[1] Ian Waudby-Smith, Lili Wu, Aaditya Ramdas, Nikos Karampatziakis, and Paul Mineiro. 2022. Anytime-valid off-policy inference for contextual bandits. *arXiv:2210.10768*. https://doi.org/10.48550/arXiv.2210.10768

No independence across nodes or windows is needed. Coverage includes saved valid nodes and subsequently selected cuts. Only saved valid nodes need computation; unobserved nodes use $S^{\mathrm{win}}_{t,G} = 0$ with the same fixed denominator.

B.5 Correctness and complexity of the tree-cut dynamic program

Fix $\mathcal{T}$, valid nodes $\mathcal{V}$, positive scores $A(G)$, and budget $m \geq 1$. Let $C_G(k)$ be valid cuts covering every leaf below G exactly once with k nodes, and let $F^\star(G,k)$ be their maximum total score, with $\max \varnothing = -\infty$. Such a cut either selects G, possible only when $G \in \mathcal{V}, k = 1$, or combines cuts from every child. The shared budget matters because a group allocated to one child is unavailable to its siblings. The recurrence compares all such allocations rather than optimizing each branch separately. Child subtrees are disjoint, so both the scores and group counts add:

$$\begin{aligned} S(G,k) &= \begin{cases} A(G), & G \in \mathcal{V},\ k = 1, \\ -\infty, & \text{otherwise}, \end{cases} \\ H^\star(G,k) &= \max_{\substack{k_1 + \cdots + k_d = k \\ k_j \geq 1}} \sum_{j=1}^{d} F^\star(G_j, k_j), \\ F^\star(G,k) &= \max\{S(G,k), H^\star(G,k)\}. \end{aligned}$$

At leaves the combination option is $-\infty$; invalid parents may still have valid descendant cuts. Postorder induction proves correctness: every feasible cut supplies a recurrence option, and every finite option is attained by a selection or the union of attaining child cuts. Thus the computed table equals $F^\star$, including infeasible states.

Combine children in a fixed order, initializing $H_0(0) = 0$ and all other entries to $-\infty$. The implementation uses

$$\begin{aligned} H_j(0) &= -\infty, \\ H_j(k) &= \max_{1 \leq s \leq k} \{H_{j-1}(k-s) + F(G_j, s)\}, \\ F(G,k) &= \max\{S(G,k), H_d(k)\}. \end{aligned}$$

Induction on j shows that H_j optimizes allocations among the first j children; truncating at m loses no allocation with total at most m. Record each maximizing allocation $J_j(k)$ and whether a state selects or combines. Starting from $t_d = k$, recover $k_j = J_j(t_j)$ and $t_{j-1} = t_j - k_j$ in reverse order, then recurse. These transitions give positive child counts summing to k, attain the stored score, and end at $t_0 = 0$. Retaining selection on a tie remains correct.

At the root, choose $k^\star = \min \arg\max_{1 \leq k \leq m} F(r,k)$. The valid installed cut guarantees a finite state. Theorem 2 converts this score optimum to optimal target gain, preferring fewer groups on ties. Each edge requires one convolution over $m(m+1)/2$ state pairs, giving $O(|\mathcal{T}|m^2)$ time. Tables, marks, and one allocation array per edge use $O(|\mathcal{T}|m)$ space; convolution uses two additional $O(m)$ arrays. Backtracking costs $O(|\mathcal{T}|)$. Scoring and subsequent handoffs are outside these bounds.

For numerical scaling, use the same offset $a = \max_{G \in \mathcal{V}} \log A(G)$ throughout the tree and $\widehat{A}(G) = e^{-a} A(G)$. Positive scaling commutes with sums and maxima, so $\widehat{F} = e^{-a} F$ on finite states; infeasible states, maximizing cuts, and tie decisions are unchanged.

Recover weights and gain by

$$w_G^\star = \frac{\widehat{A}(G)}{\sum_{B\in\Pi^\star}\widehat{A}(B)},$$
$$C^\star(\Pi^\star) = \eta^{-1}\big(a + \log \widehat{F}(r, k^\star)\big) - U^-.$$

B.6 Oracle comparison for the executed policy

Fix a window with a valid current cut. Let $\mathcal{C}$ contain valid cuts with at most m groups, and let the oracle $(\Pi^{\mathrm{or}}, w^{\mathrm{or}})$ maximize $J(q_{\Pi,w})$ over $\Pi \in \mathcal{C}$ and the full group simplex $\Delta_\Pi = \{w \geq 0 : \sum_G w_G = 1\}$. It uses the same realized contribution field, without handoff floors or KL-budget constraints. Let $\Pi^\star$ maximize $C^\star$ over $\mathcal{C}$, and let $\Pi^{\mathrm{exec}} \in \mathcal{C}$ be installed. On Keep, these installed quantities are Π^-, $w_G^{\mathrm{exec}} = q^-(G)$, and $q^{\mathrm{exec}} = q^-$. The loss from the installed cut relative to the searched cut is $\Gamma = C^\star(\Pi^\star) - C^\star(\Pi^{\mathrm{exec}}) \geq 0$. Let $w^{\star,\mathrm{exec}}$ be the target on the installed cut. Its execution loss is $R^{\mathrm{exec}} = \eta^{-1} D_{\mathrm{KL}}(w^{\mathrm{exec}} \| w^{\star,\mathrm{exec}})$. By Equation (A.4), this equals $C^\star(\Pi^{\mathrm{exec}}) - C(\Pi^{\mathrm{exec}}, w^{\mathrm{exec}})$.

The oracle gap separates cut choice, execution, and forecast error:

$$J(q^{\mathrm{or}}) - J(q^{\mathrm{exec}}) \leq \Gamma + R^{\mathrm{exec}} + \sum_{G\in\Pi^{\mathrm{or}}} w_G^{\mathrm{or}}[2\beta(G) + e(G)] + \sum_{G\in\Pi^{\mathrm{exec}}} w_G^{\mathrm{exec}} e(G).$$

Indeed, write $J(q_{\Pi,w}) = C(\Pi, w) + U^- + \rho(\Pi, w)$, where $\rho(\Pi, w) = \sum_G w_G[v(G) - L(G)]$. The nodewise bounds $-e(G) \leq v(G) - L(G) \leq 2\beta(G) + e(G)$ control the two residuals. Target optimality gives $C(\Pi^{\mathrm{or}}, w^{\mathrm{or}}) \leq C^\star(\Pi^{\mathrm{or}}) \leq C^\star(\Pi^\star)$; subtracting the executed score and canceling U^- proves the bound. On joint coverage of both cuts, all their $e(G)$ vanish, leaving $\Gamma + R^{\mathrm{exec}} + 2\sum_{G\in\Pi^{\mathrm{or}}} w_G^{\mathrm{or}}\beta(G)$. Installing the searched cut additionally makes $\Gamma = 0$.